Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman bin Faisal University
College of Science and Humanities in Jubail
Computer science department

المملكة العربية السعودية
وزارة التعليم
جامعة الإمام عبدالرحمن بن فيصل
كلية العلوم و الدراسات الإنسانية بالجبيل
قسم علوم الحاسب الالي

Project Report

Project Name: Oka veterinary BB Group: 04

1 Project Description:

Oka Veterinary Clinics is a store allocated in Eastern region in Saudi Arabia ,and now we improve a Website serves as a vital platform for pet owners, with its management structured around two key roles: administrators and super administrators. This hierarchical framework facilitates efficient service delivery, ensuring that both users and staff are effectively supported. By clearly defining the responsibilities of each user type, the website enhances its mission to promote animal welfare and streamline pet care.

User Type 1: Regular User

Regular users, primarily pet owners, gain access to a comprehensive range of services on the Oka Veterinary Clinics Website, designed to simplify the management of their pets' needs. They can easily book essential services such as grooming, medical check-ups, and vaccinations through an intuitive appointment scheduling system. The website also features an e-commerce section for purchasing pet supplies, including food, toys, and accessories. Informational pages, such as "About Us" and "Contact Us," provide critical insights into the clinic's mission and operations. Additionally, users can engage in the adoption process by viewing available pets and submitting applications to adopt or rehome animals, thereby fostering community involvement in animal welfare.

User Type 2: Administrator

Administrators are integral to the operation of the Oka Veterinary Clinics Website, tasked with managing users and ensuring a seamless experience. They oversee the registration and management of regular users, addressing inquiries and resolving any issues that may arise. Administrators also manage the platform's service offerings, ensuring that information is accurate and up to date. They facilitate appointment bookings and maintain the e-commerce shop, ensuring that product

listings reflect current inventory. Furthermore, administrators handle adoption requests, ensuring that all processes align with best practices in animal welfare, thereby supporting the clinic's commitment to responsible pet ownership.

User Type 3: Super Administrator

Super administrators hold the highest level of authority on the Oka Veterinary Clinics Website, managing both administrators and regular users. Their responsibilities include overseeing the administrative team, providing support and training to ensure effective user management. Super administrators also manage veterinary staff and ensure consistent

service delivery across multiple clinic branches, if applicable. They are responsible for maintaining the accuracy of the e-commerce inventory and ensuring that all aspects of the website function smoothly. By overseeing both users and administrators, super administrators play a critical role in enhancing operational efficiency and supporting the clinic's mission to care for pets and their owners.

In summary, the Oka Veterinary Clinics Website is systematically structured to address the diverse needs of its user base, which comprises regular users, administrators, and super administrators. By clearly defining the roles and responsibilities of each user type, the platform enhances operational efficiency and fosters a community dedicated to animal welfare. This comprehensive approach ensures that high-quality services are provided, responsible pet ownership is promoted, and meaningful connections between pets and potential adopters are facilitated, ultimately supporting the clinic's mission to care for pets and their owners.

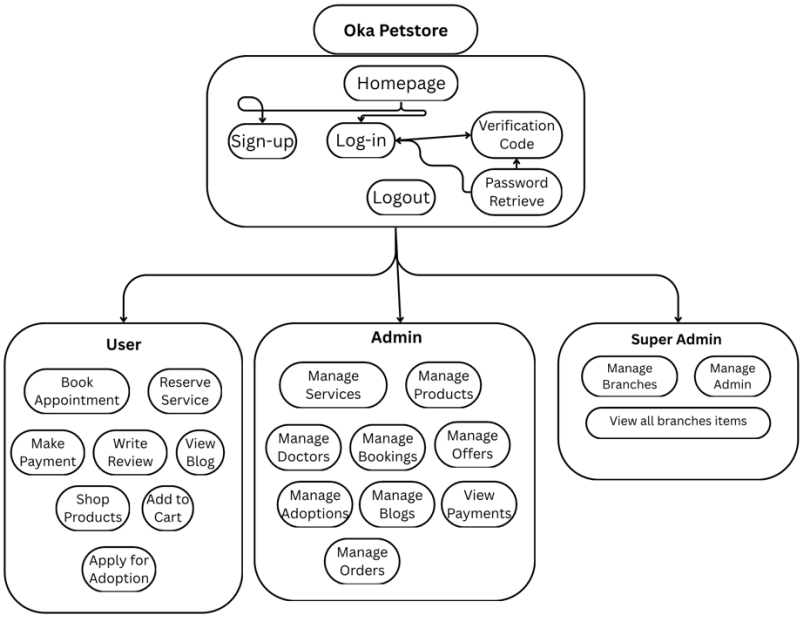
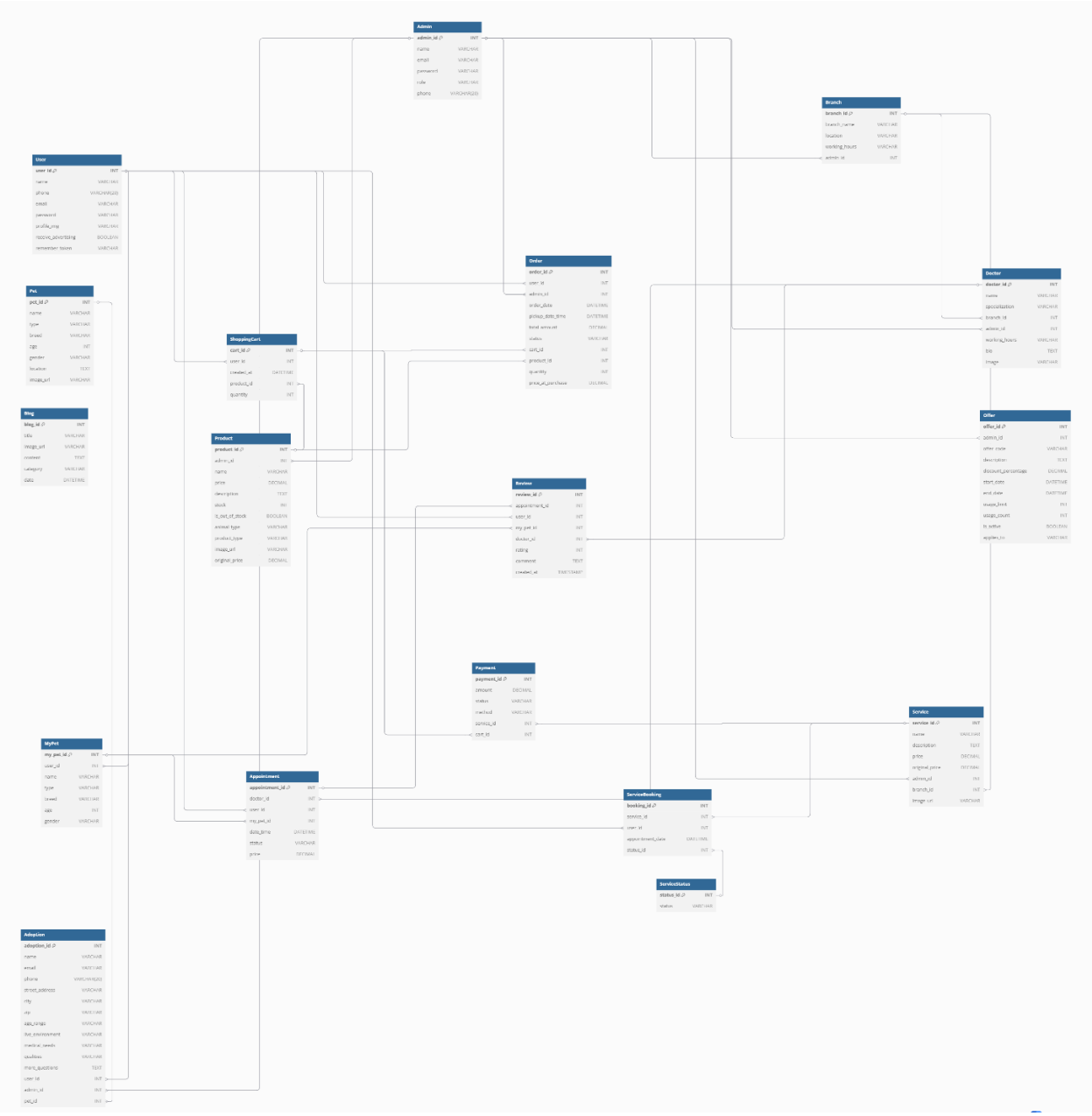


Figure 1 System Architecture

2 Database design + tables + ER design

2.1 ER design:



~~2.1~~2.2 SQL codes :

```
CREATE TABLE User (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255),
  phone VARCHAR(20),
  email VARCHAR(255),
  password VARCHAR(255),
  profile_img VARCHAR(255),
  receive_advertising BOOLEAN,
  remember_token VARCHAR(255)
);
CREATE TABLE Admin (
  admin_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255),
  email VARCHAR(255),
  password VARCHAR(255),
  role ENUM('admin', 'super') NOT NULL,
  phone VARCHAR(20)
);
CREATE TABLE Branch (
  branch_id INT PRIMARY KEY AUTO_INCREMENT,
  branch_name VARCHAR(255),
  location VARCHAR(255),
  working_hours VARCHAR(255),
```

```
  admin_id INT,
  FOREIGN KEY (admin_id) REFERENCES Admin(admin_id)
);
CREATE TABLE Doctor (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255),
  specialization VARCHAR(255),
  branch_id INT,
  admin_id INT,
  working_hours VARCHAR(255),
  bio TEXT,
  image VARCHAR(255),
  FOREIGN KEY (branch_id) REFERENCES Branch(branch_id),
  FOREIGN KEY (admin_id) REFERENCES Admin(admin_id)
);
CREATE TABLE Product (
  product_id INT PRIMARY KEY AUTO_INCREMENT,
  admin_id INT,
  name VARCHAR(255),
  price DECIMAL(10,2),
  description TEXT,
  stock INT,
  is_out_of_stock BOOLEAN,
  animal_type ENUM('Dog', 'Cat', 'Bird', 'Small Animal'),
```

```
  product_type ENUM('Food', 'Toy', 'Supplies'),
  image_url VARCHAR(255),
  original_price DECIMAL(10,2),
  FOREIGN KEY (admin_id) REFERENCES Admin(admin_id)
);
CREATE TABLE ShoppingCart (
  cart_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  created_at DATETIME,
  product_id INT,
  quantity INT,
  FOREIGN KEY (user_id) REFERENCES User(user_id),
  FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
CREATE TABLE `Order` (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  admin_id INT,
  order_date DATETIME,
  pickup_date_time DATETIME,
  total_amount DECIMAL(10,2),
  status ENUM('Pending', 'Confirmed', 'Cancelled', 'Completed'),
  cart_id INT,
  product_id INT,
```

```
  quantity INT,
  price_at_purchase DECIMAL(10,2),
  FOREIGN KEY (user_id) REFERENCES User(user_id),
  FOREIGN KEY (admin_id) REFERENCES Admin(admin_id),
  FOREIGN KEY (cart_id) REFERENCES ShoppingCart(cart_id),
  FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
CREATE TABLE Pet (
  pet_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255),
  type VARCHAR(255),
  breed VARCHAR(255),
  age INT,
  gender ENUM('Male', 'Female'),
  location TEXT,
  ImageURL VARCHAR(255)
);
CREATE TABLE MyPet (
  my_pet_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  name VARCHAR(100),
  type VARCHAR(100),
  breed VARCHAR(100),
  age INT,
```

```
  gender ENUM('Male', 'Female'),
  FOREIGN KEY (user_id) REFERENCES User(user_id)
);
CREATE TABLE Appointment (
  appointment_id INT PRIMARY KEY AUTO_INCREMENT,
  doctor_id INT,
  user_id INT,
  my_pet_id INT,
date_time DATETIME,
  status ENUM('Pending', 'Cancelled', 'Completed', 'Missed'),
  price DECIMAL(10,2),
  FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id),
  FOREIGN KEY (user_id) REFERENCES User(user_id),
  FOREIGN KEY (my_pet_id) REFERENCES MyPet(my_pet_id)
);
CREATE TABLE Review (
  review_id INT PRIMARY KEY AUTO_INCREMENT,
  appointment_id INT,
  user_id INT,
  my_pet_id INT,
  doctor_id INT,
  rating INT,
  comment TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
  FOREIGN KEY (appointment_id) REFERENCES Appointment(appointment_id),
  FOREIGN KEY (user_id) REFERENCES User(user_id),
  FOREIGN KEY (my_pet_id) REFERENCES MyPet(my_pet_id),
  FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id)
);
CREATE TABLE Service (
  service_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255),
  description TEXT,
price DECIMAL(10,2),
  original_price DECIMAL(10,2),
  admin_id INT,
  branch_id INT,
  image_url VARCHAR(255),
  FOREIGN KEY (admin_id) REFERENCES Admin(admin_id),
  FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
);
CREATE TABLE ServiceStatus (
  status_id INT PRIMARY KEY AUTO_INCREMENT,
  status ENUM('Pending', 'Completed', 'Canceled')
);
CREATE TABLE ServiceBooking (
  booking_id INT PRIMARY KEY AUTO_INCREMENT,
  service_id INT,
```

```
  user_id INT,
  appointment_date DATETIME,
  status_id INT,
  FOREIGN KEY (service_id) REFERENCES Service(service_id),
  FOREIGN KEY (user_id) REFERENCES User(user_id),
  FOREIGN KEY (status_id) REFERENCES ServiceStatus(status_id)
);
CREATE TABLE Payment (
  payment_id INT PRIMARY KEY AUTO_INCREMENT,
  amount DECIMAL(10,2),
  status VARCHAR(20),
  method VARCHAR(20),
  service_id INT,
  cart_id INT,
  FOREIGN KEY (service_id) REFERENCES Service(service_id),
  FOREIGN KEY (cart_id) REFERENCES ShoppingCart(cart_id)
);
CREATE TABLE Blog (
  blog_id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(255),
  image_url VARCHAR(255),
  content TEXT,
  category VARCHAR(255),
  date DATETIME
```

```
);
CREATE TABLE Offer (
  offer_id INT PRIMARY KEY AUTO_INCREMENT,
  admin_id INT,
  offer_code VARCHAR(100) UNIQUE,
  description TEXT,
  discount_percentage DECIMAL(5,2),
  start_date DATETIME,
  end_date DATETIME,
  usage_limit INT,
  usage_count INT,
  is_active BOOLEAN,
  applies_to ENUM('product', 'service', 'all'),
 FOREIGN KEY (admin_id) REFERENCES Admin(admin_id)
);
CREATE TABLE Adoption (
  adoption_id INT PRIMARY KEY AUTO_INCREMENT,
  Name VARCHAR(255),
  Email VARCHAR(255),
  Phone VARCHAR(20),
  StreetAddress VARCHAR(255),
  City VARCHAR(100),
  Zip VARCHAR(20),
  Age VARCHAR(50),
```

```
  Live VARCHAR(50), -- مثل inside, outdoor
  MedicalNeeds VARCHAR(10), -- yes / no
  Qualities VARCHAR(255), -- مثل active
  MoreQuestions TEXT,
  FOREIGN KEY (user_id) REFERENCES User(user_id),
  FOREIGN KEY (admin_id) REFERENCES Admin(admin_id),
  FOREIGN KEY (pet_id) REFERENCES Pet(pet_id)
)
```

2.3. Tables:

2.3.1. User

Column Name	Data Type	Constraints
user_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(255)	
phone	VARCHAR(20)	
email	VARCHAR(255)	
password	VARCHAR(255)	
profile_img	VARCHAR(255)	
receive_advertising	BOOLEAN	

remember_token	VARCHAR(255)	

2.3.2. Admin

Column Name	Data Type	Constraints
admin_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(255)	
email	VARCHAR(255)	
password	VARCHAR(255)	
role	ENUM('admin','super')	NOT NULL
phone	VARCHAR(20)	

2.3.3. Branch

Column Name	Data Type	Constraints
branch_id	INT	PRIMARY KEY, AUTO_INCREMENT
branch_name	VARCHAR(255)	
location	VARCHAR(255)	

working_hours	VARCHAR(255)	
admin_id	INT	FOREIGN KEY REFERENCES Admin(admin_id)

2.3.4. Do=ctor

Column Name	Data Type	Constraints
doctor_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(255)	
specialization	VARCHAR(255)	
branch_id	INT	FOREIGN KEY REFERENCES Branch(branch_id)
admin_id	INT	FOREIGN KEY REFERENCES Admin(admin_id)
working_hours	VARCHAR(255)	
bio	TEXT	
image	VARCHAR(255)	

2.3.5. Product

Column Name	Data Type	Constraints
product_id	INT	PRIMARY KEY, AUTO_INCREMENT

admin_id	INT	FOREIGN KEY REFERENCES Admin(admin_id)
name	VARCHAR(255)	
price	DECIMAL(10,2)	
description	TEXT	
stock	INT	
is_out_of_stock	BOOLEAN	
animal_type	ENUM('Dog', 'Cat', 'Bird', 'Small Animal')	
product_type	ENUM('Food', 'Toy', 'Supplies')	
image_url	VARCHAR(255)	
original_price	DECIMAL(10,2)	

2.3.6. ShoppingCart

Column Name	Data Type	Constraints
cart_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FOREIGN KEY REFERENCES User(user_id)
created_at	DATETIME	
product_id	INT	FOREIGN KEY REFERENCES Product(product_id)
quantity	INT	

2.3.7. Order

Column Name	Data Type	Constraints
order_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FOREIGN KEY REFERENCES User(user_id)
admin_id	INT	FOREIGN KEY REFERENCES Admin(admin_id)
order_date	DATETIME	
pickup_date_time	DATETIME	
total_amount	DECIMAL(10,2)	
status	ENUM('Pending', 'Confirmed', 'Cancelled', 'Completed')	
cart_id	INT	FOREIGN KEY REFERENCES ShoppingCart(cart_id)
product_id	INT	FOREIGN KEY REFERENCES Product(product_id)
quantity	INT	
price_at_purchase	DECIMAL(10,2)	

2.3.8. Pet

Column Name	Data Type	Constraints
pet_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT

name	VARCHAR(255)	
type	VARCHAR(255)	
breed	VARCHAR(255)	
age	INT(11)	
gender	ENUM('Male','Female')	
location	TEXT	
ImageURL	VARCHAR(255)	

2.3.9. MyPet

Column Name	Data Type	Constraints
my_pet_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FOREIGN KEY REFERENCES User(user_id)
name	VARCHAR(100)	
type	VARCHAR(100)	
breed	VARCHAR(100)	
age	INT	
gender	ENUM('Male','Female')	

2.3.10. Appointment

Column Name	Data Type	Constraints
appointment_id	INT	PRIMARY KEY, AUTO_INCREMENT
doctor_id	INT	FOREIGN KEY REFERENCES

		Doctor(doctor_id)
user_id	INT	FOREIGN KEY REFERENCES User(user_id)
my_pet_id	INT	FOREIGN KEY REFERENCES MyPet(my_pet_id)
date_time	DATETIME	
status	ENUM('Pending', 'Cancelled', 'Completed', 'Missed')	
price	DECIMAL(10,2)	

2.3.11. Review

Column Name	Data Type	Constraints
review_id	INT	PRIMARY KEY, AUTO_INCREMENT
appointment_id	INT	FOREIGN KEY REFERENCES Appointment(appointment_id)
user_id	INT	FOREIGN KEY REFERENCES User(user_id)
my_pet_id	INT	FOREIGN KEY REFERENCES MyPet(my_pet_id)
doctor_id	INT	FOREIGN KEY REFERENCES Doctor(doctor_id)
rating	INT	
comment	TEXT	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

2.3.12. Service

Column Name	Data Type	Constraints
service_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(255)	
description	TEXT	
price	DECIMAL(10,2)	
original_price	DECIMAL(10,2)	
admin_id	INT	FOREIGN KEY REFERENCES Admin(admin_id)
branch_id	INT	FOREIGN KEY REFERENCES Branch(branch_id)
image_url	VARCHAR(255)	

2.3.13. serviceStatus

Column Name	Data Type	Constraints
status_id	INT	PRIMARY KEY, AUTO_INCREMENT
status	ENUM('Pending','Completed','Canceled')	

2.3.14. ServiceBooking

Column Name	Data Type	Constraints
booking_id	INT	PRIMARY KEY, AUTO_INCREMENT
service_id	INT	FOREIGN KEY REFERENCES

		Service(service_id)
user_id	INT	FOREIGN KEY REFERENCES User(user_id)
appointment_date	DATETIME	
status_id	INT	FOREIGN KEY REFERENCES ServiceStatus(status_id)

2.3.15. Payment

Column Name	Data Type	Constraints
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT
amount	DECIMAL(10,2)	
status	VARCHAR(20)	
method	VARCHAR(20)	
service_id	INT	FOREIGN KEY REFERENCES Service(service_id)
cart_id	INT	FOREIGN KEY REFERENCES ShoppingCart(cart_id)

2.3.16. Blog

Column Name	Data Type	Constraints
blog_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT
title	VARCHAR(255)	

image_url	VARCHAR(255)	
content	TEXT	
category	VARCHAR(255)	
date	DATETIME	

3. Offer

Column Name	Data Type	Constraints
offer_id	INT	PRIMARY KEY, AUTO_INCREMENT
admin_id	INT	FOREIGN KEY REFERENCES Admin(admin_id)
offer_code	VARCHAR(100)	UNIQUE
description	TEXT	
discount_percentage	DECIMAL(5,2)	
start_date	DATETIME	
end_date	DATETIME	
usage_limit	INT	
usage_count	INT	
is_active	BOOLEAN	
applies_to	ENUM('product', 'service', 'all')	

3.3.1. Adoption

Column Name	Data Type	Constraints
adoption_id	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(255)	
Email	VARCHAR(255)	
Phone	VARCHAR(20)	
StreetAddress	VARCHAR(255)	
City	VARCHAR(100)	
Zip	VARCHAR(20)	
Age	VARCHAR(20)	
Live	VARCHAR(50)	e.g. inside/outdoor
MedicalNeeds	VARCHAR(50)	yes / no
Qualities	VARCHAR(255)	
MoreQuestions	TEXT	

4. Web application Interface.

4.3. Users interfaces

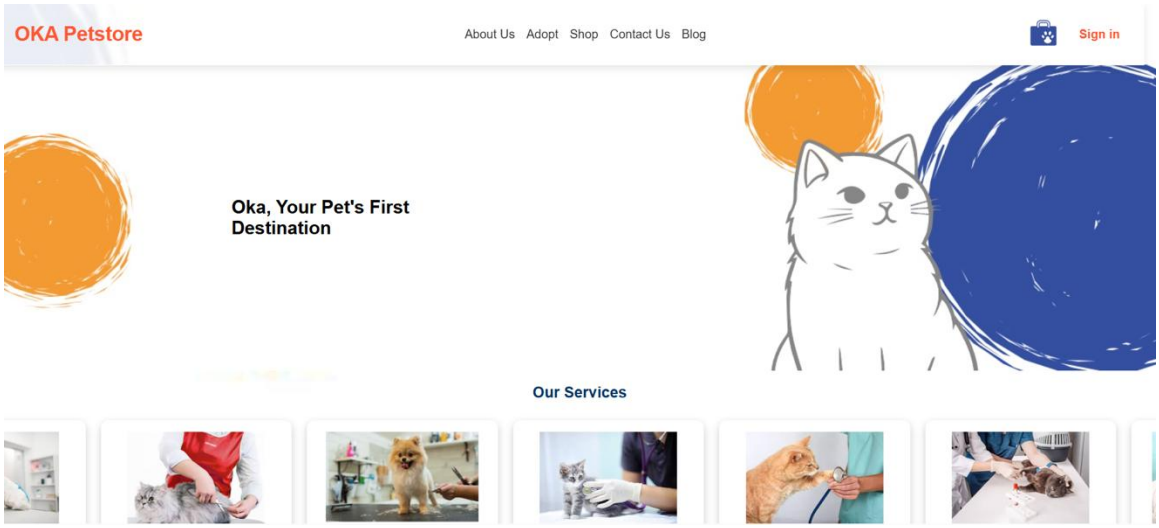


Figure 2 home page before user registered

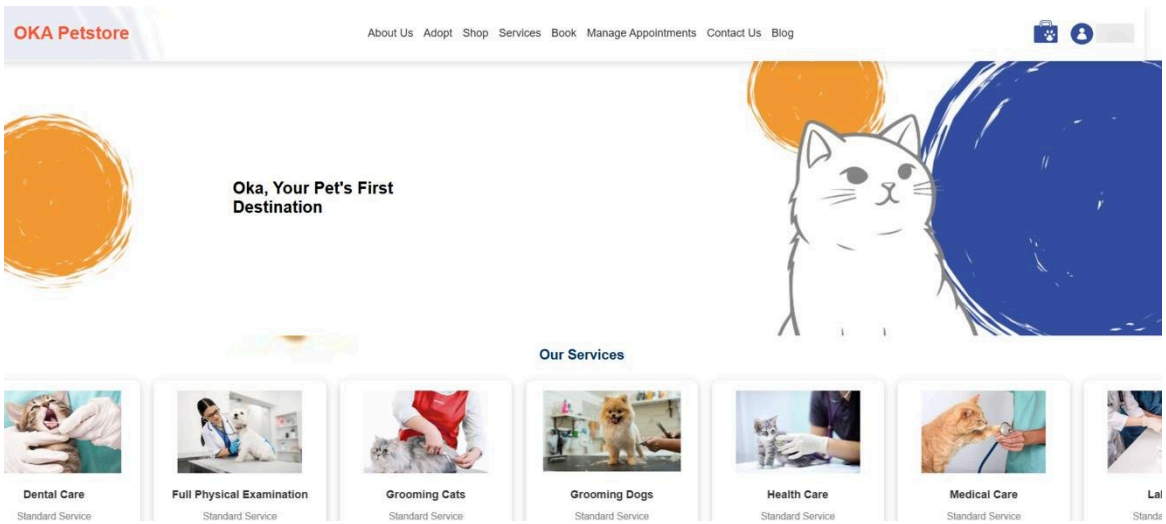


Figure 3 home page after user registered

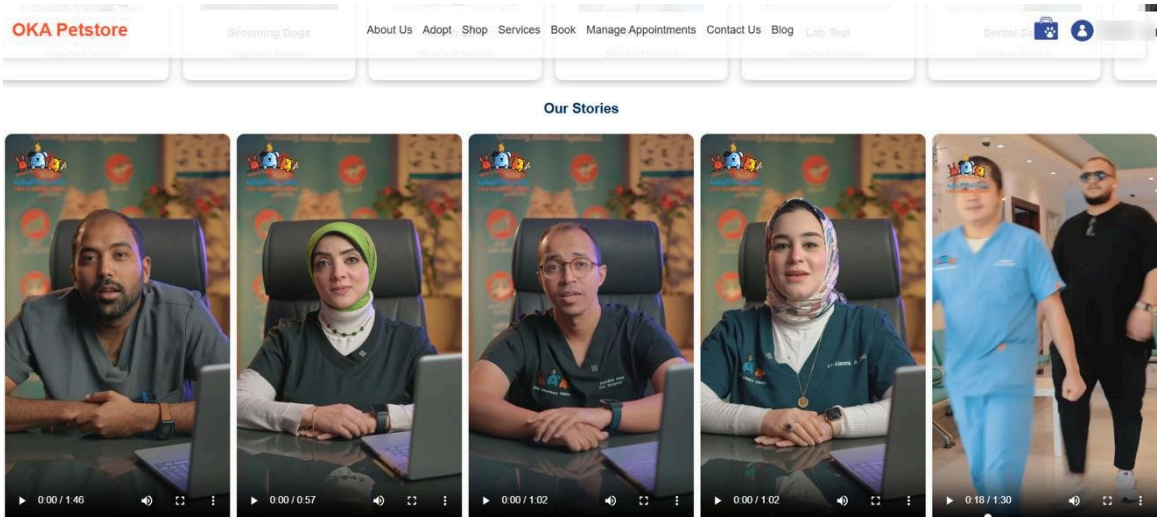


Figure 4 home page after user registered

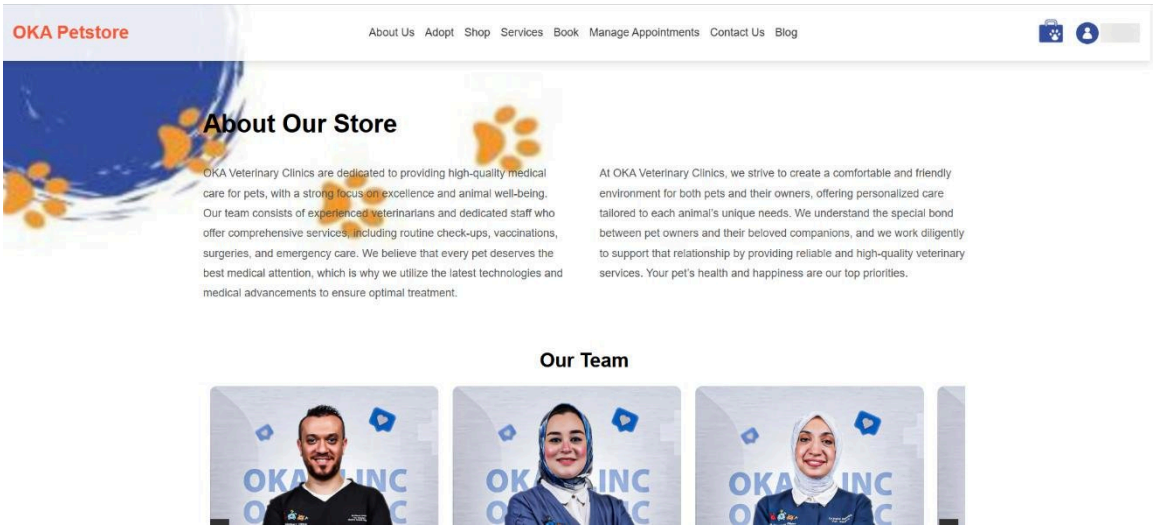


Figure 5 about us

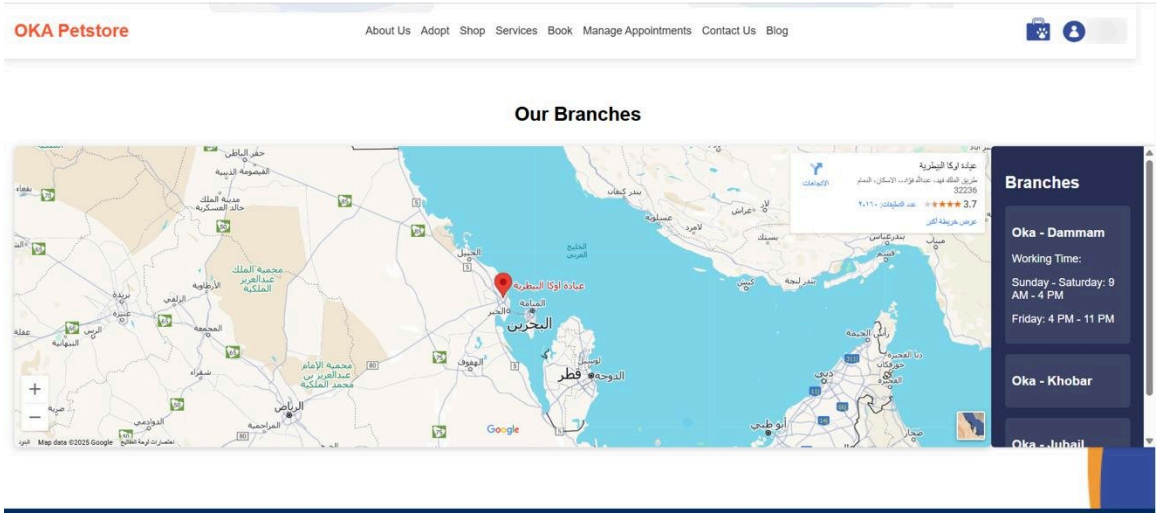


Figure 6 API interactive map

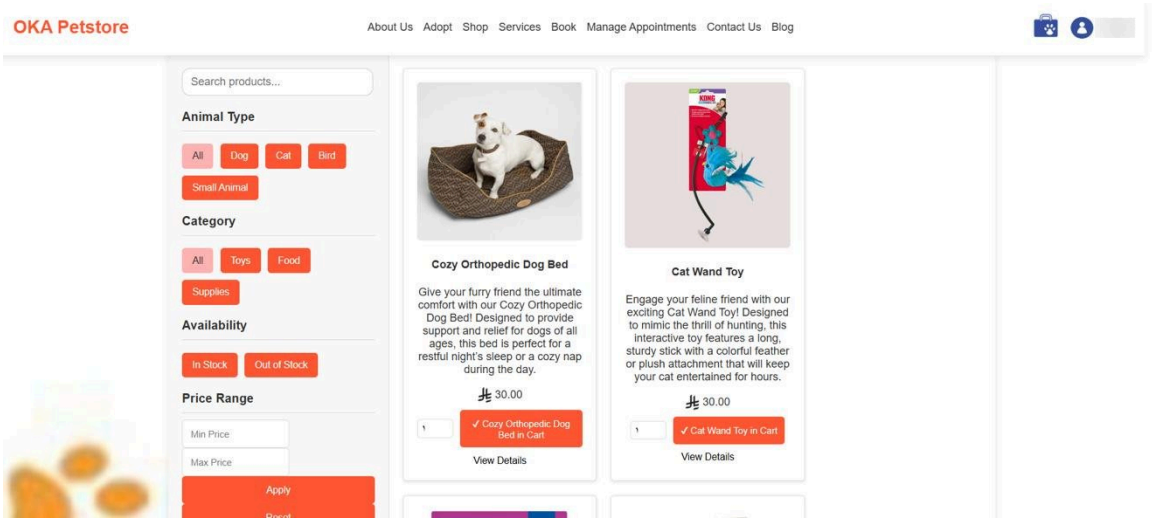


Figure 7 Shop page

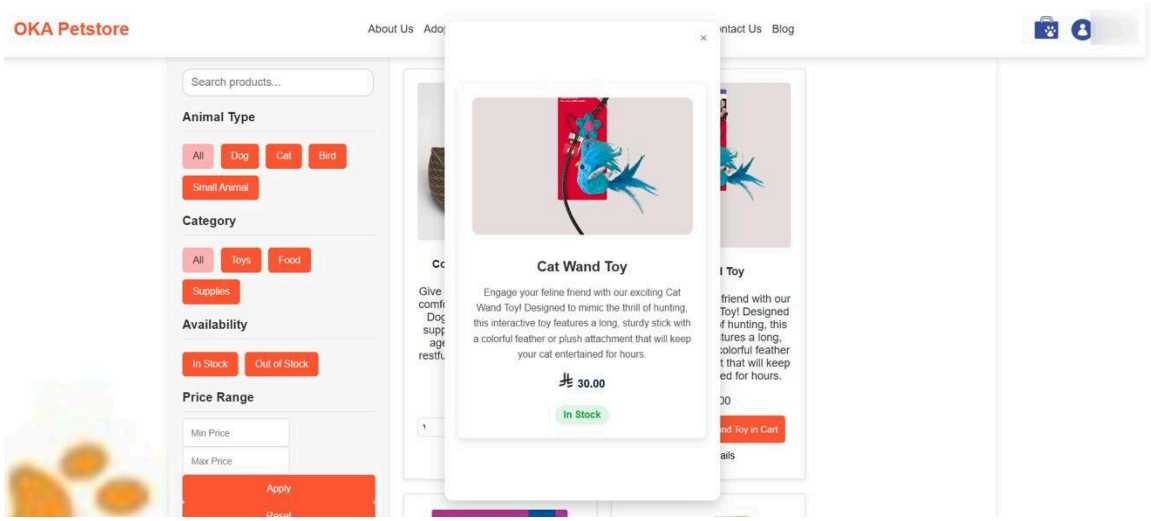


Figure 8 shop page - product info

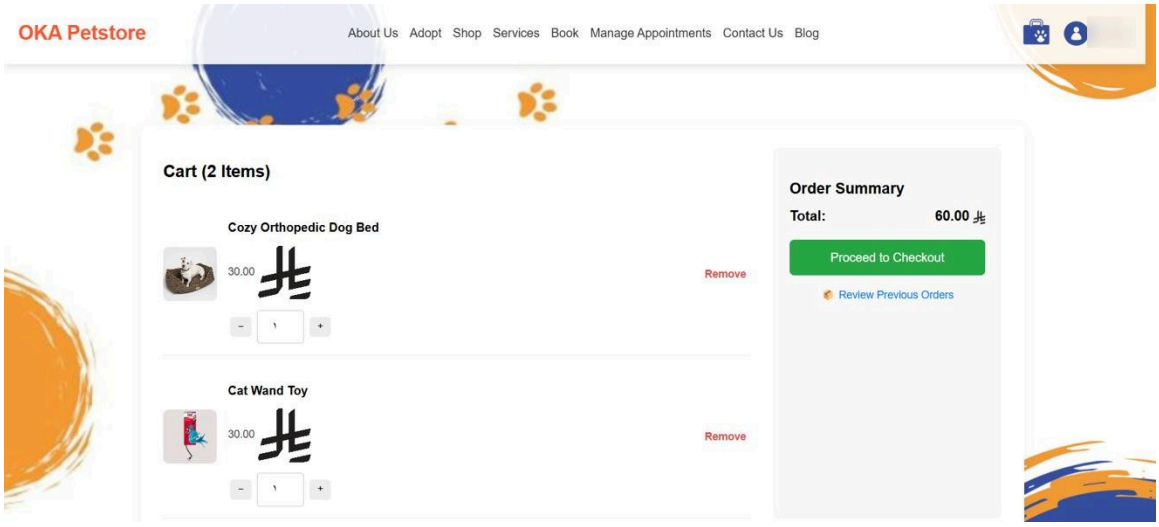


Figure 9 cart

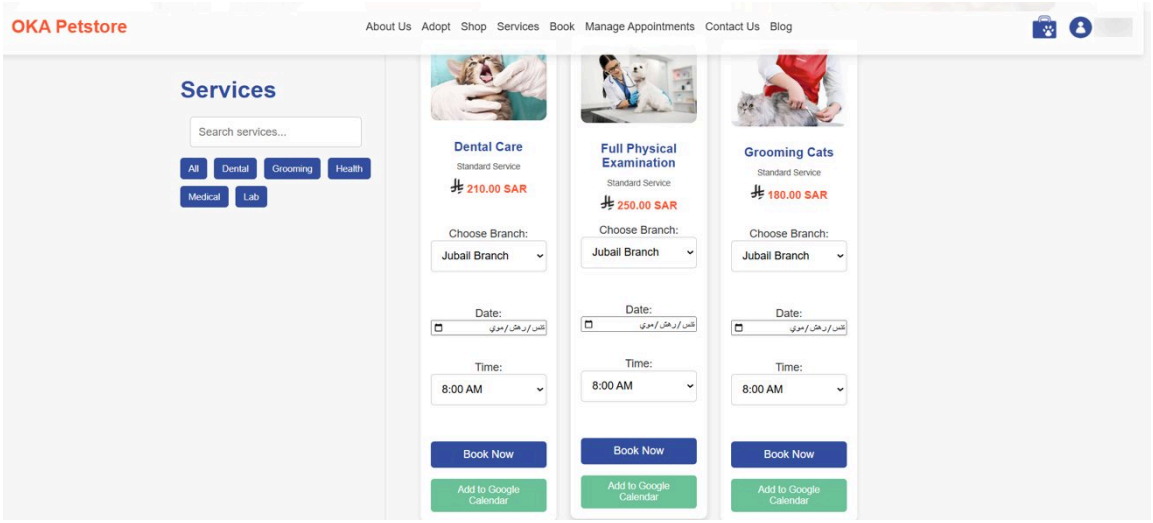


Figure 10 services

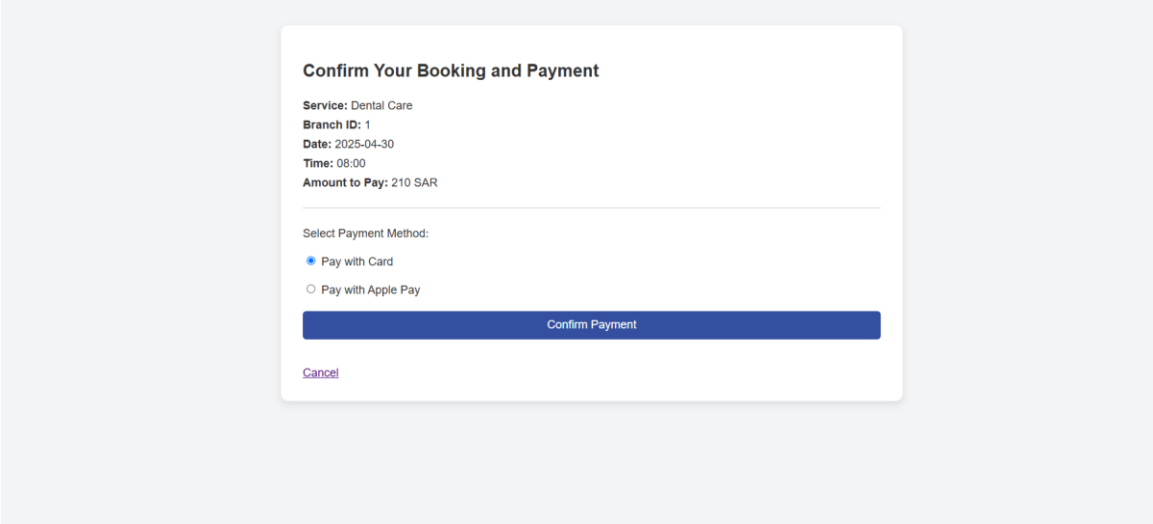


Figure 11 services booking

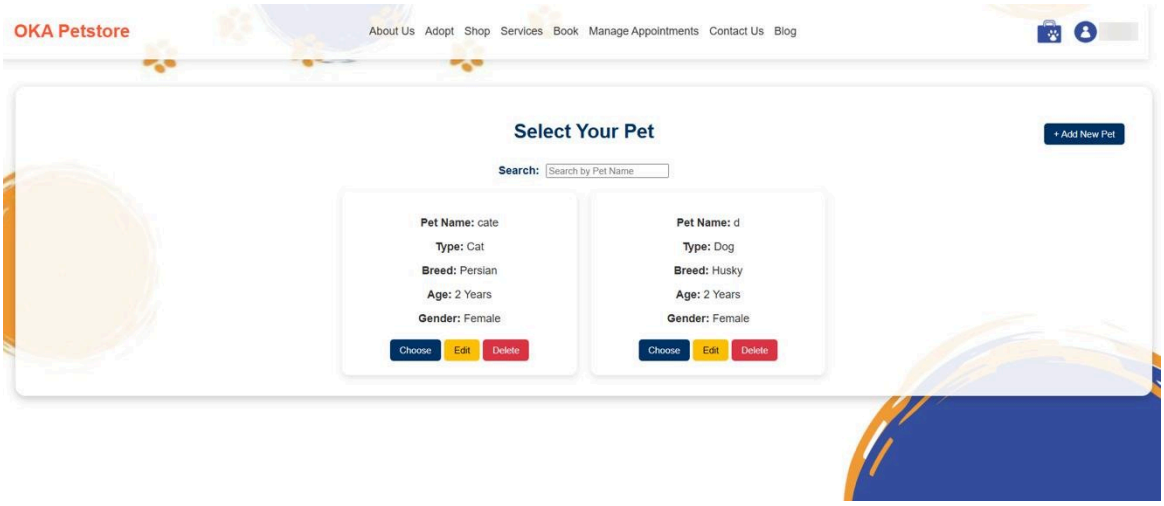


Figure 12 book appointments

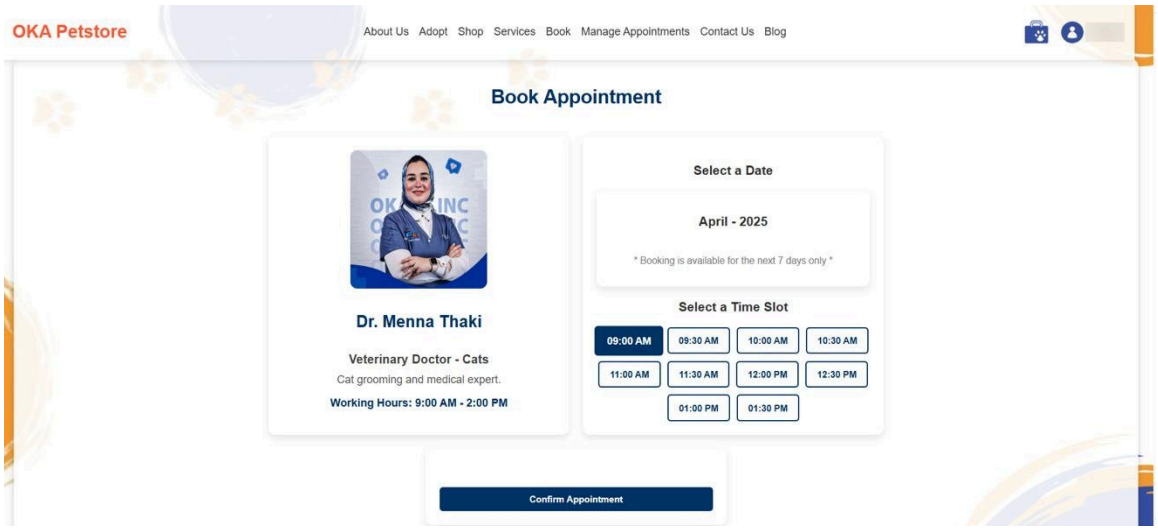


Figure 13 book appointments - book doctor –

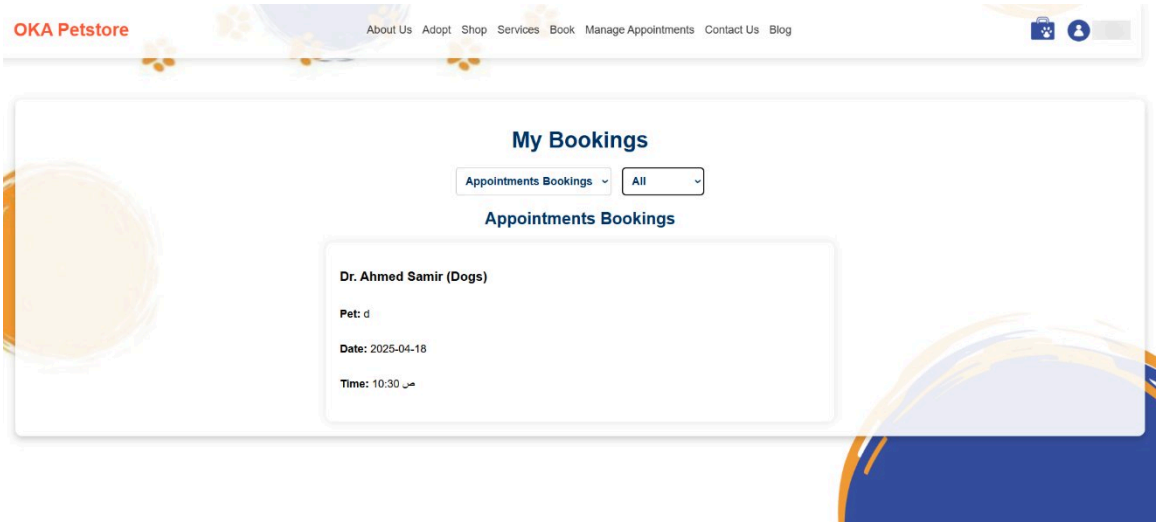


Figure 14 mange appointments

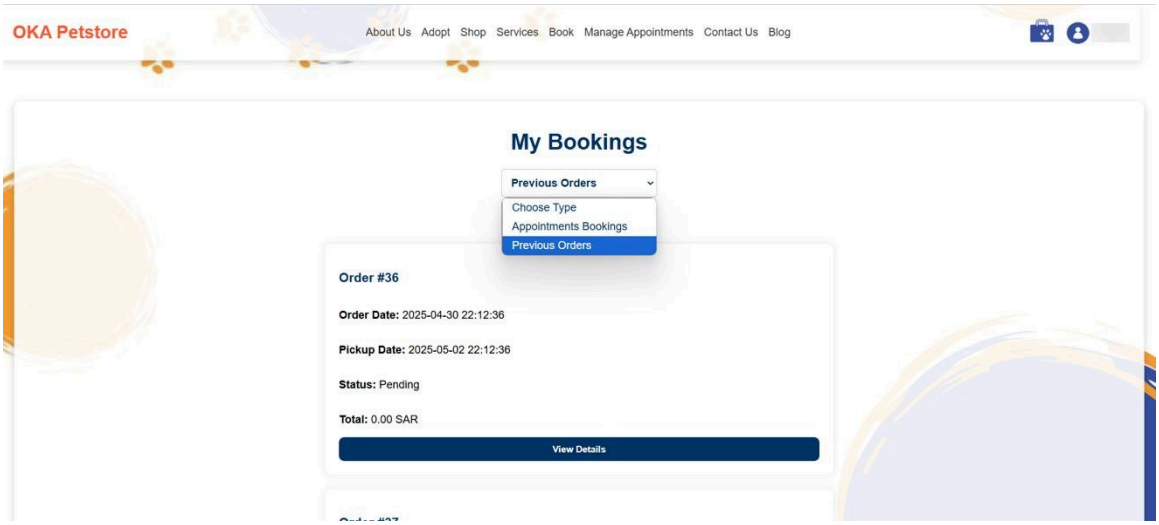


Figure 15 previous orders

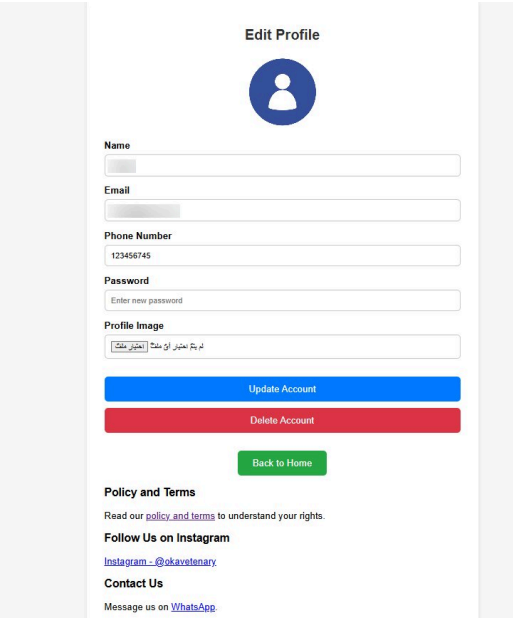


Figure 16 user profile

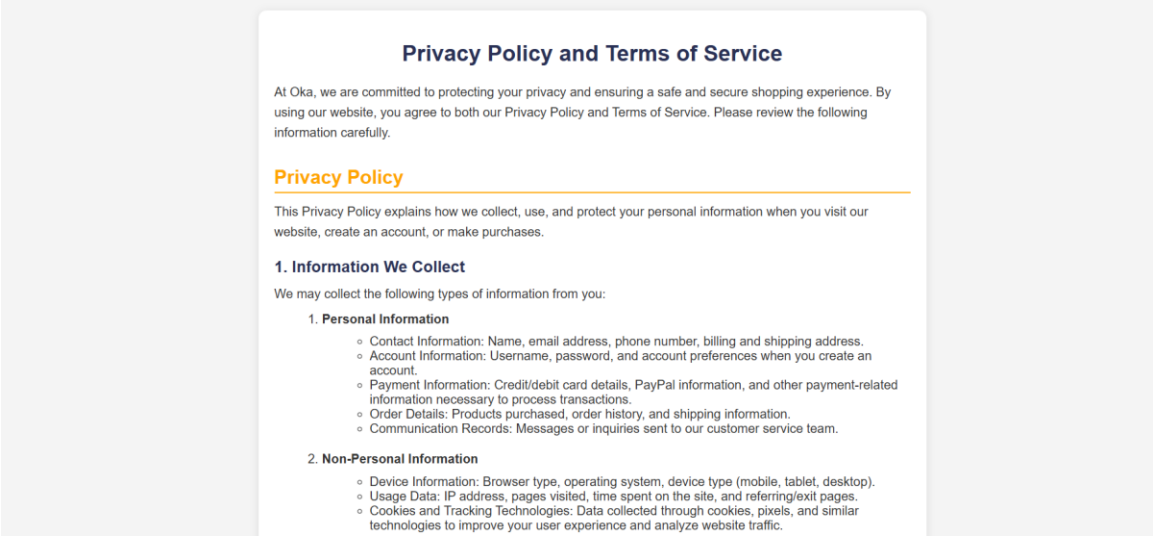


Figure 17 privacy policy and terms of service

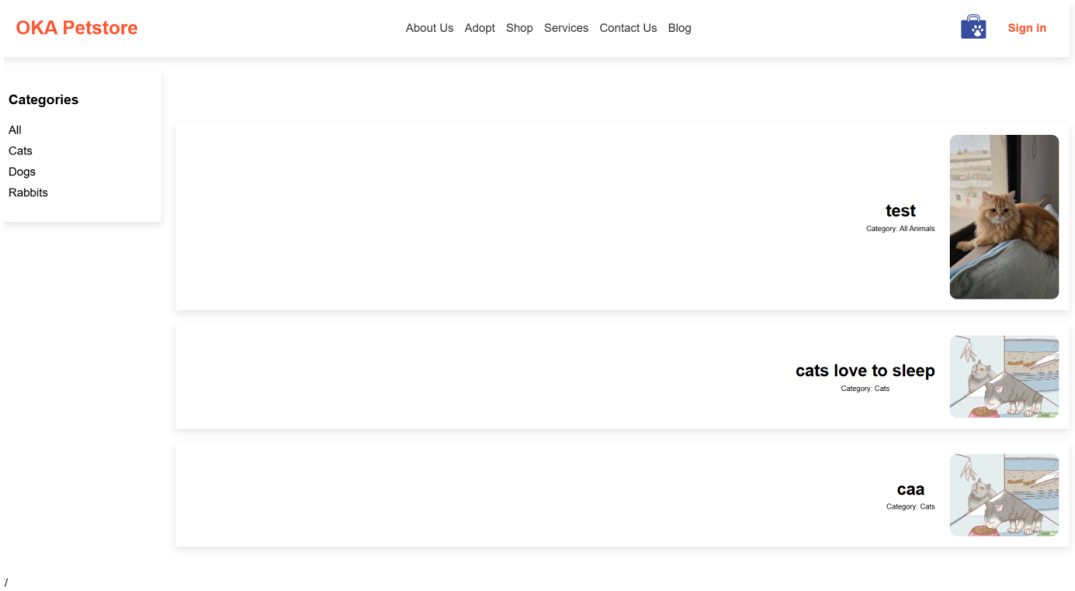


Figure 18 blogs

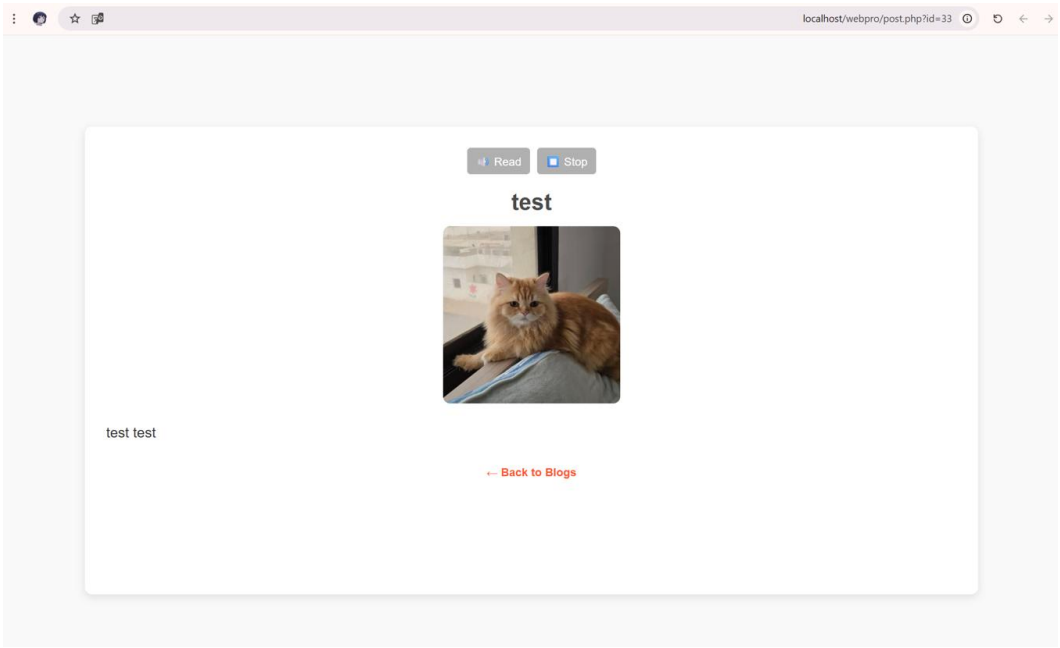


Figure 19 post

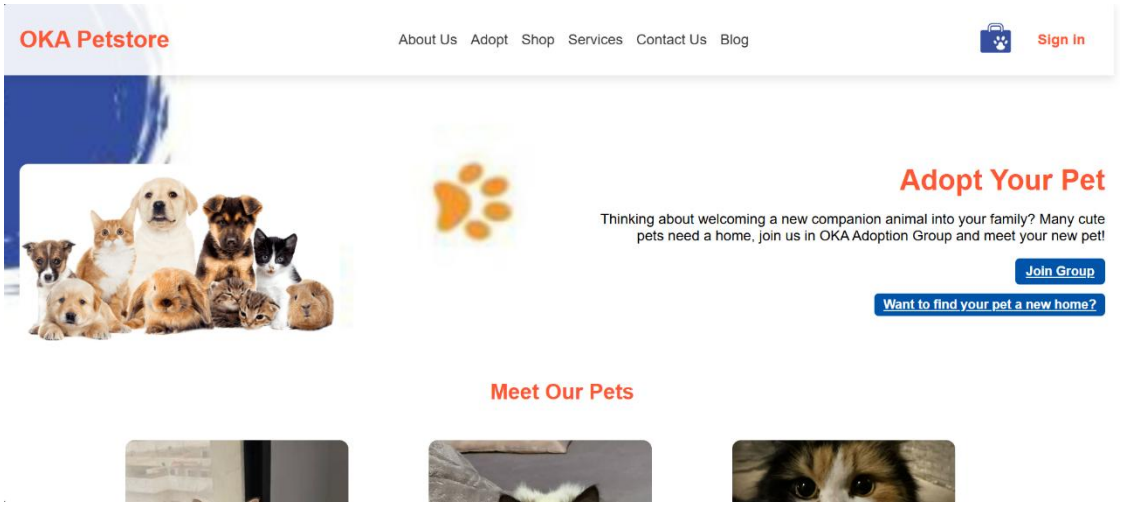


Figure 20 Adopt

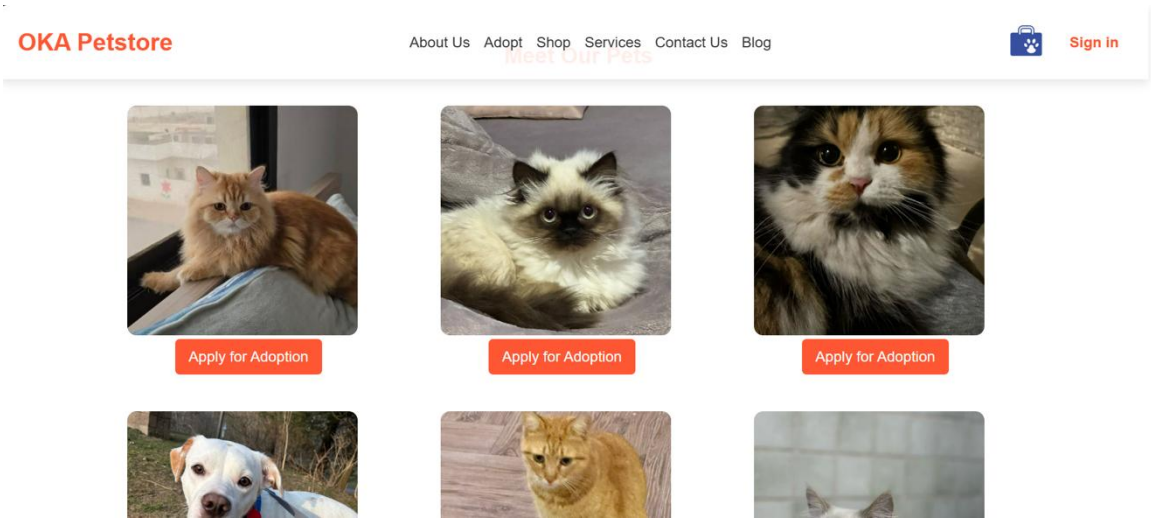


Figure 21 Adopt 2

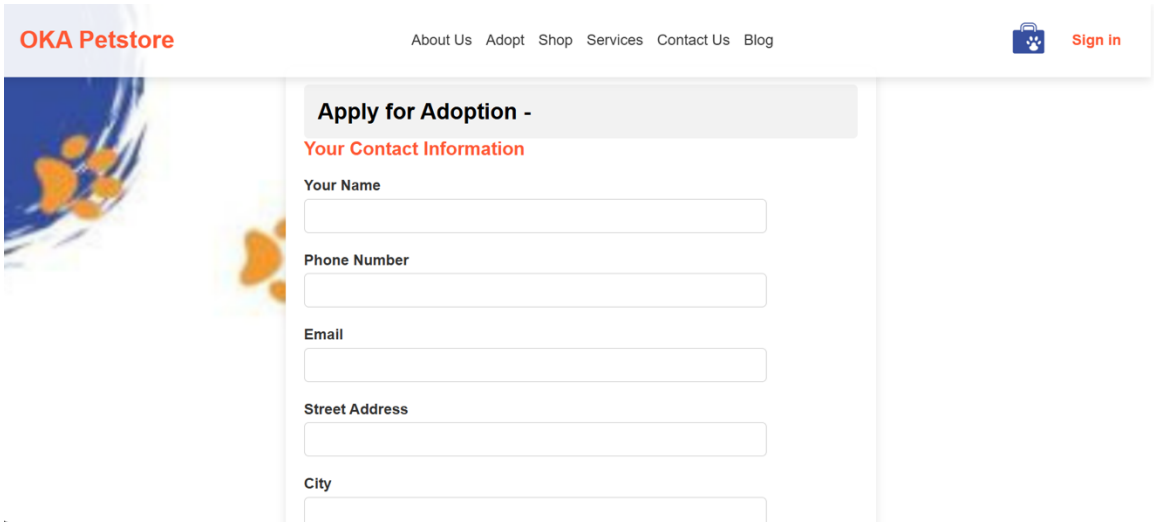


Figure 22 Adoption Application

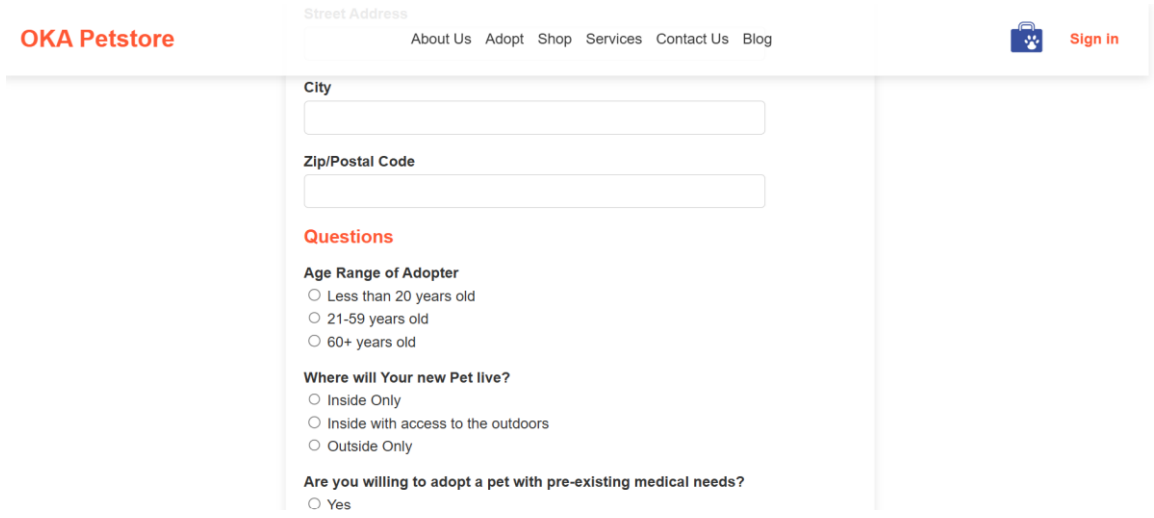


Figure 23 Adoption Application 2

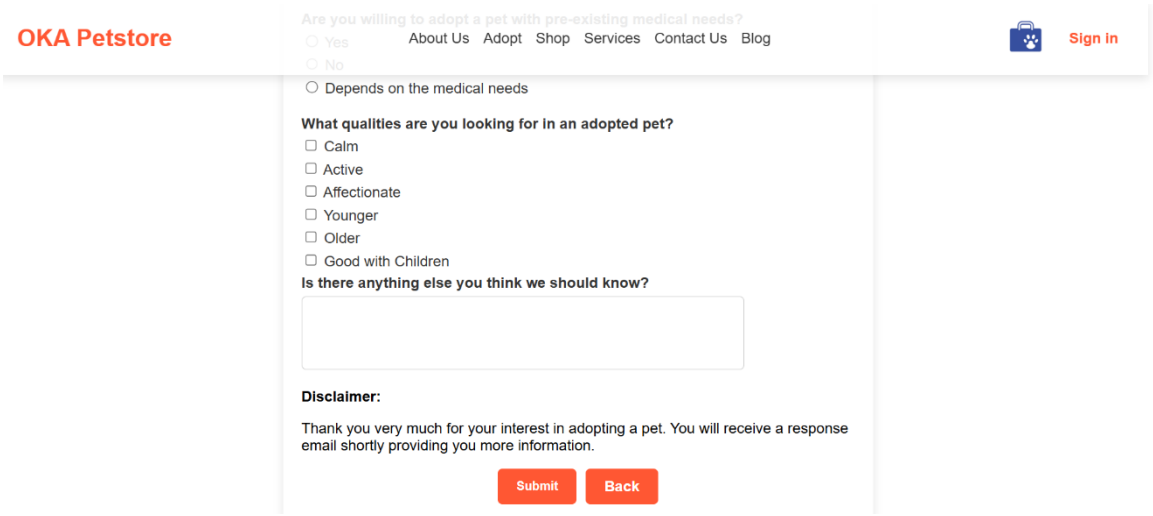


Figure 24 Adoption Application 3

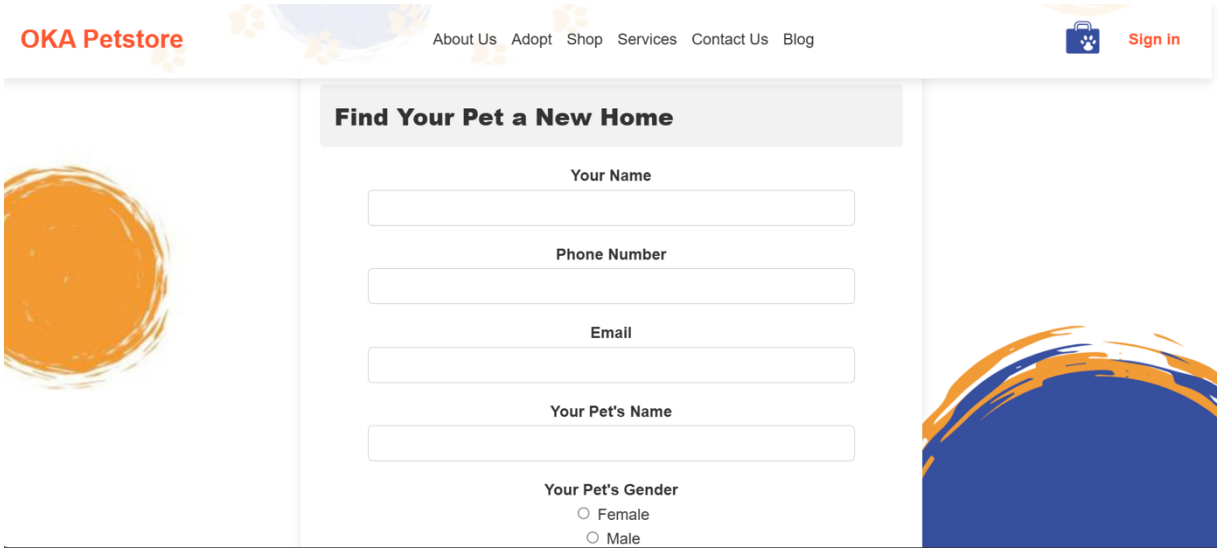


Figure 25 Find Your Pet a New Home

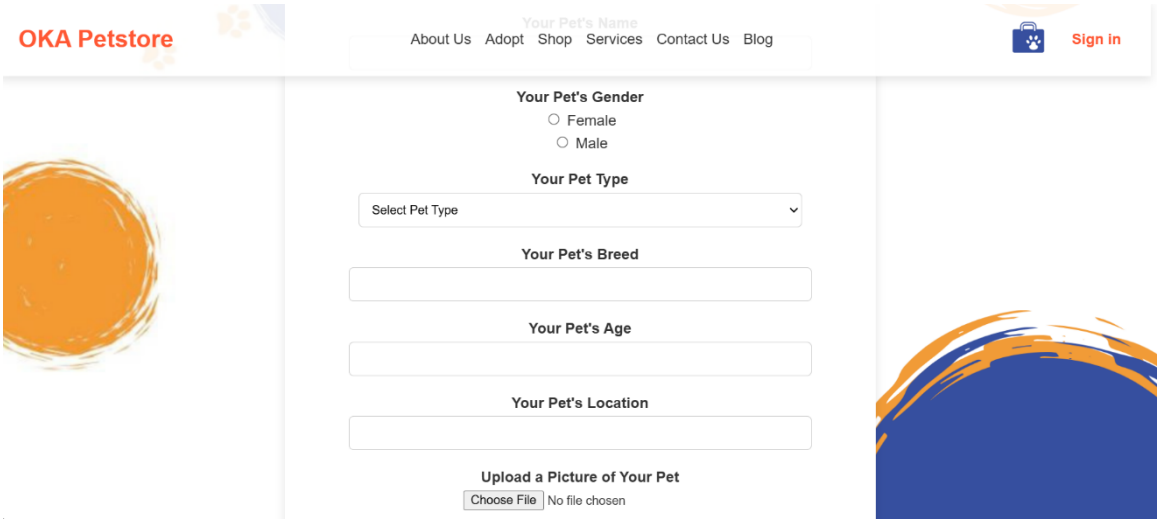


Figure 26 Find Your Pet a New Home 2

4.4. admin interfaces:

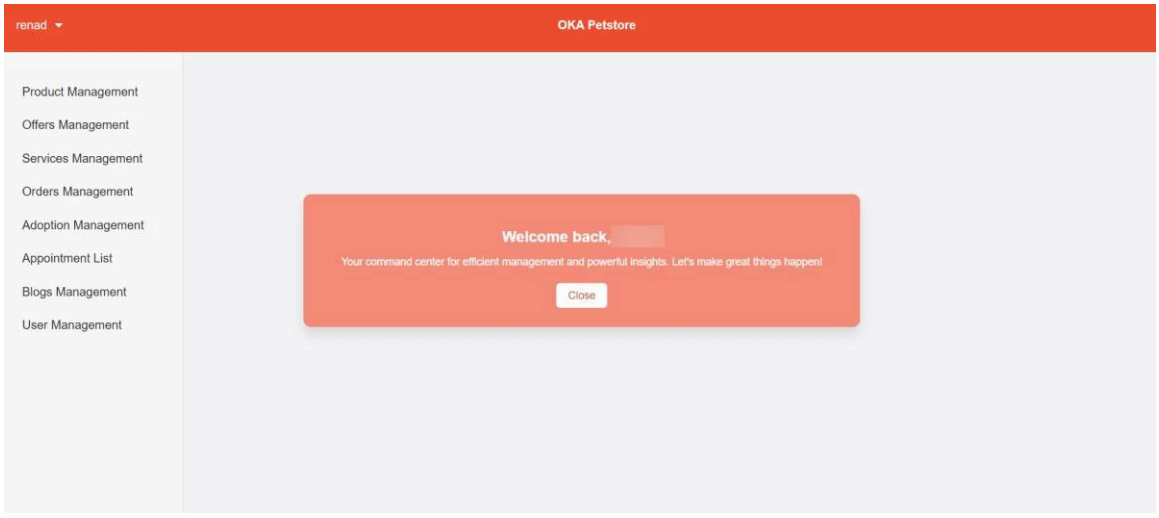


Figure 27 admin welcome page

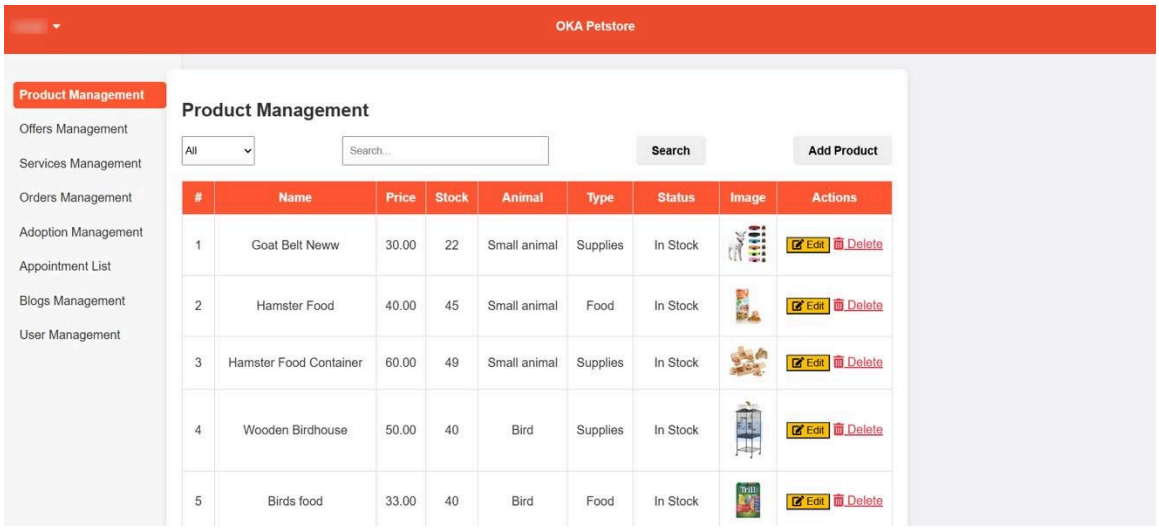


Figure 28 product management

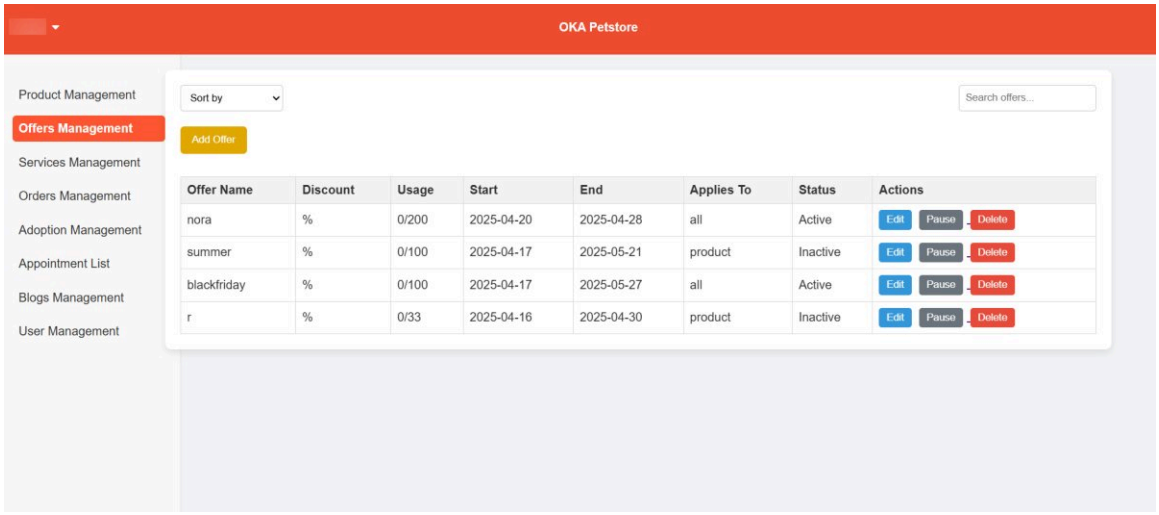


Figure 29 offers management

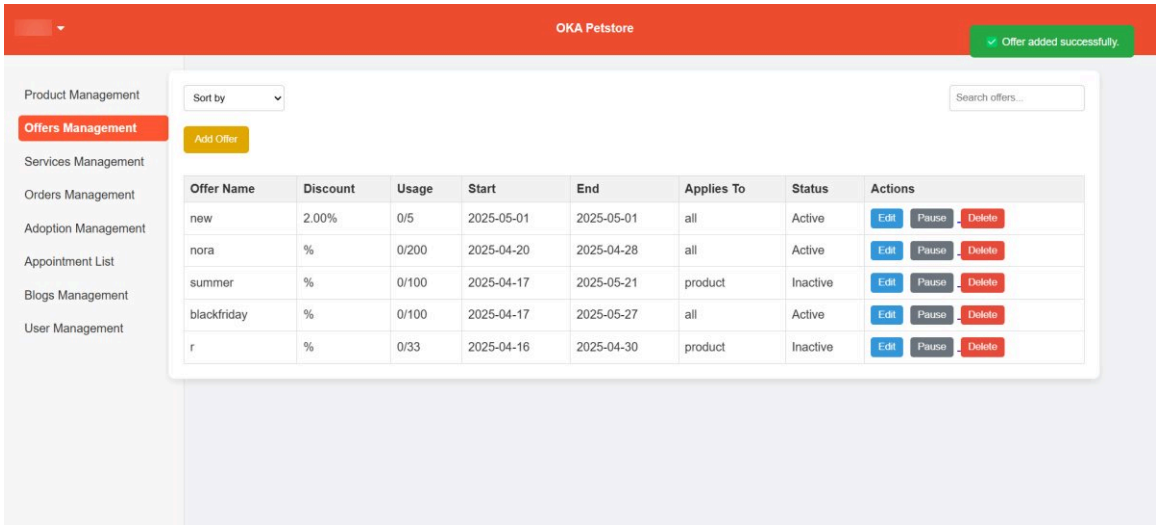


Figure 30 offers management

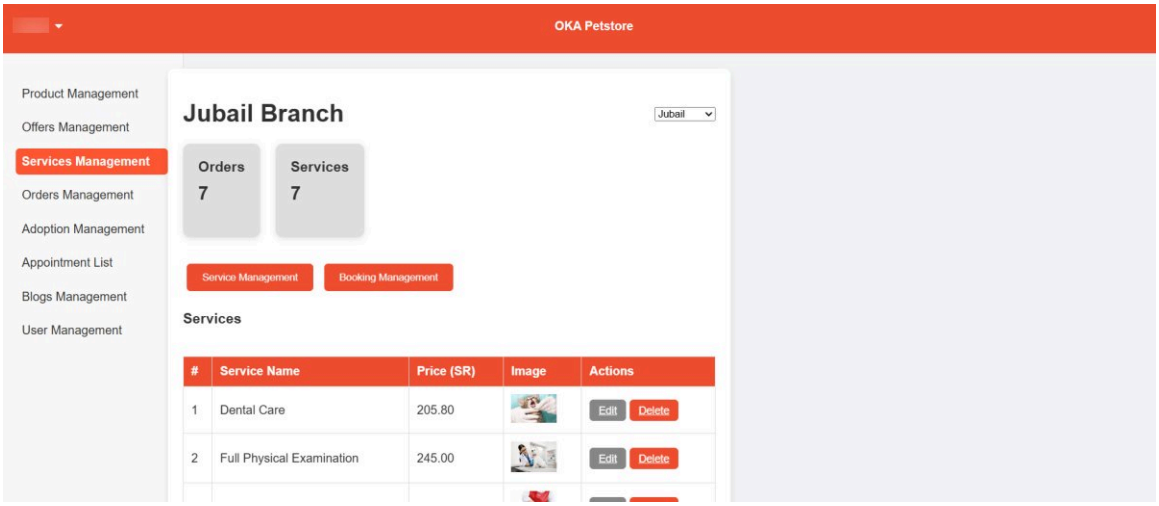


Figure 31 services management

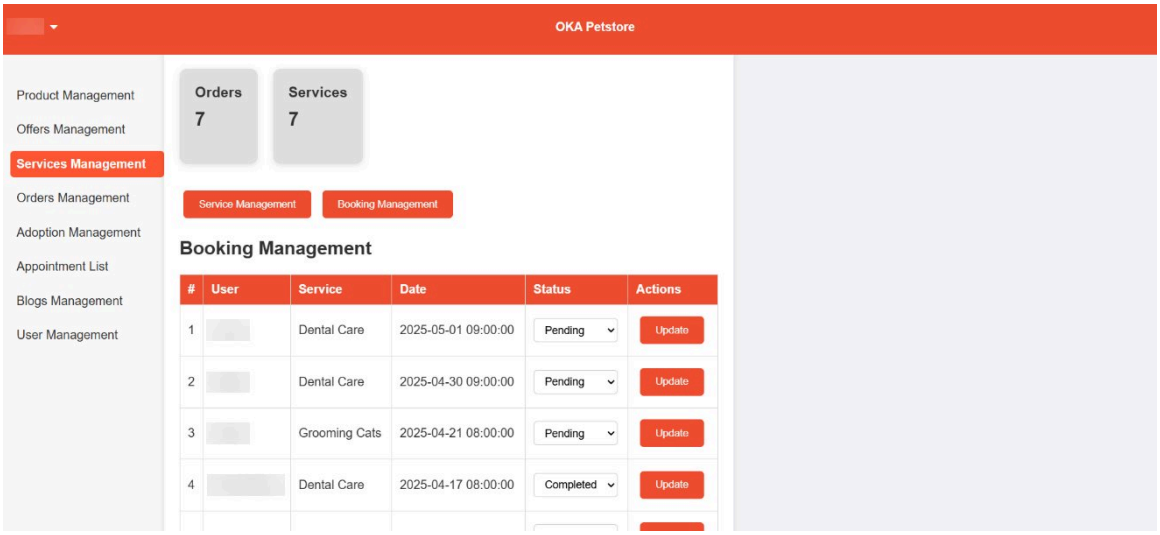


Figure 32 services management 2

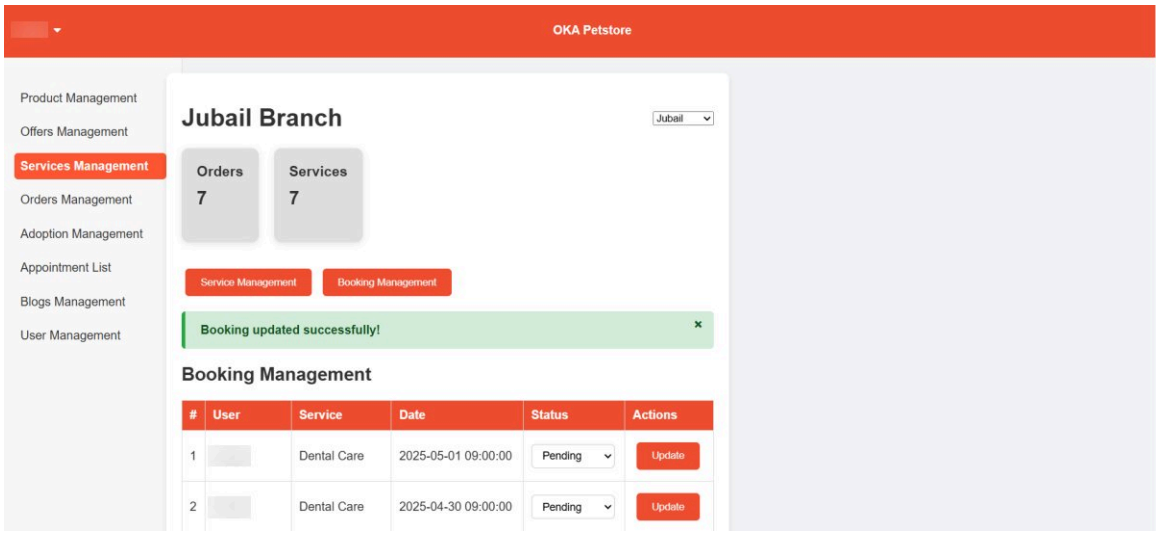


Figure 33 services management 3

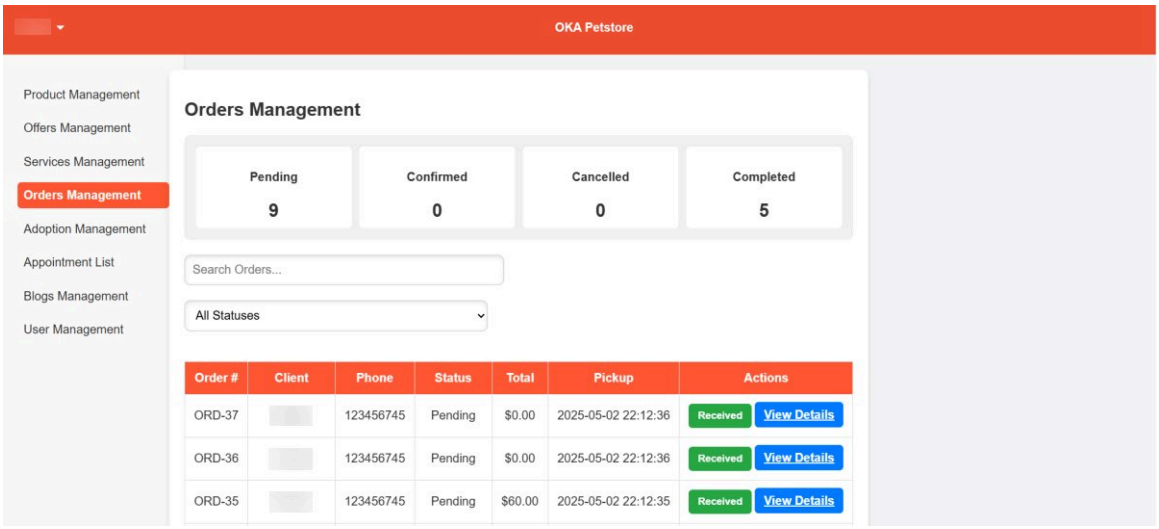


Figure 34 orders management

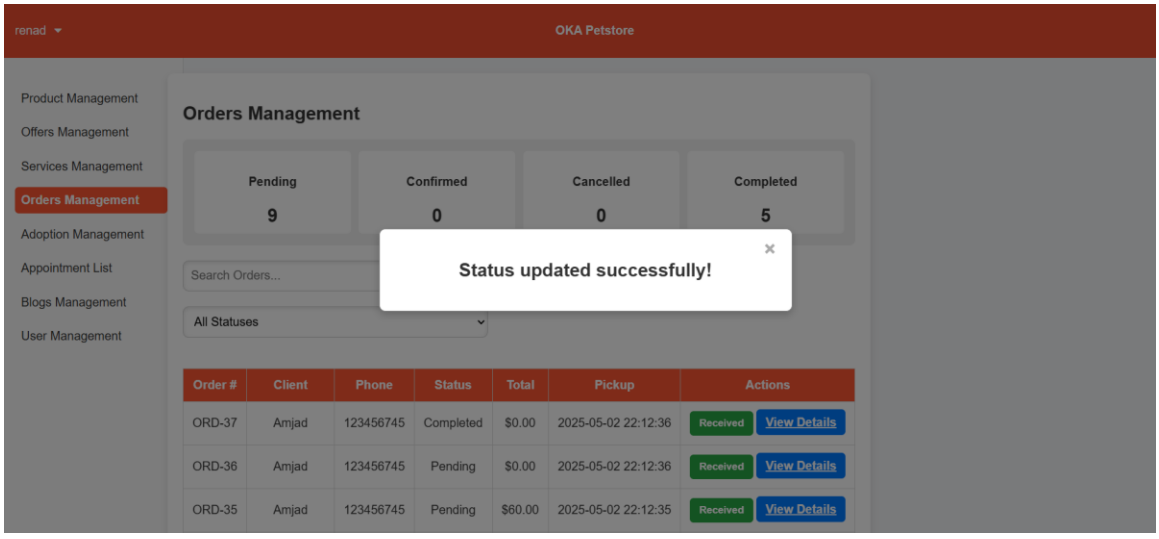


Figure 35 order management 2

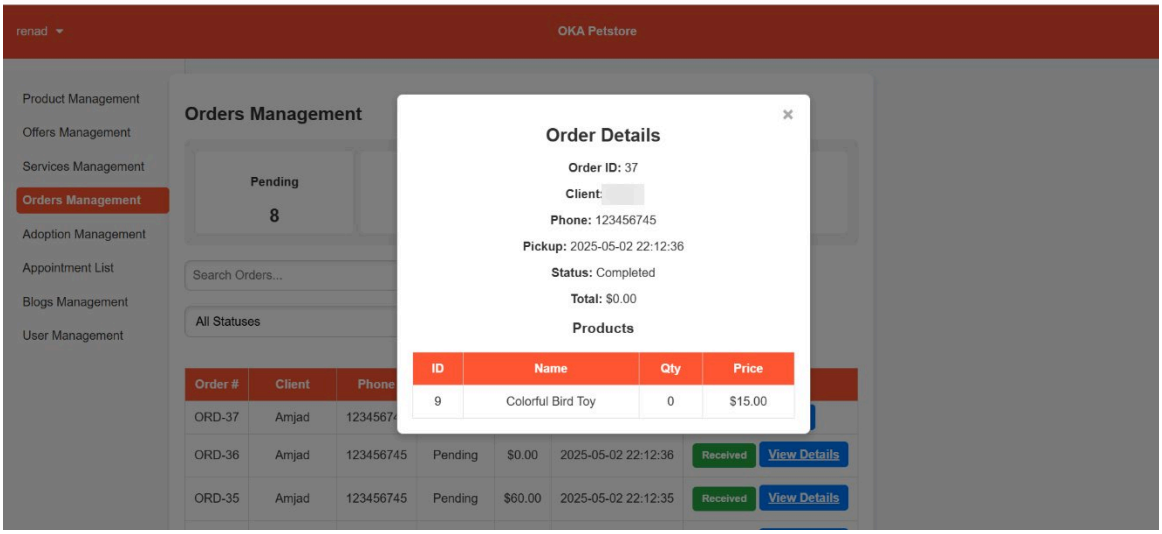


Figure 35 order management 3

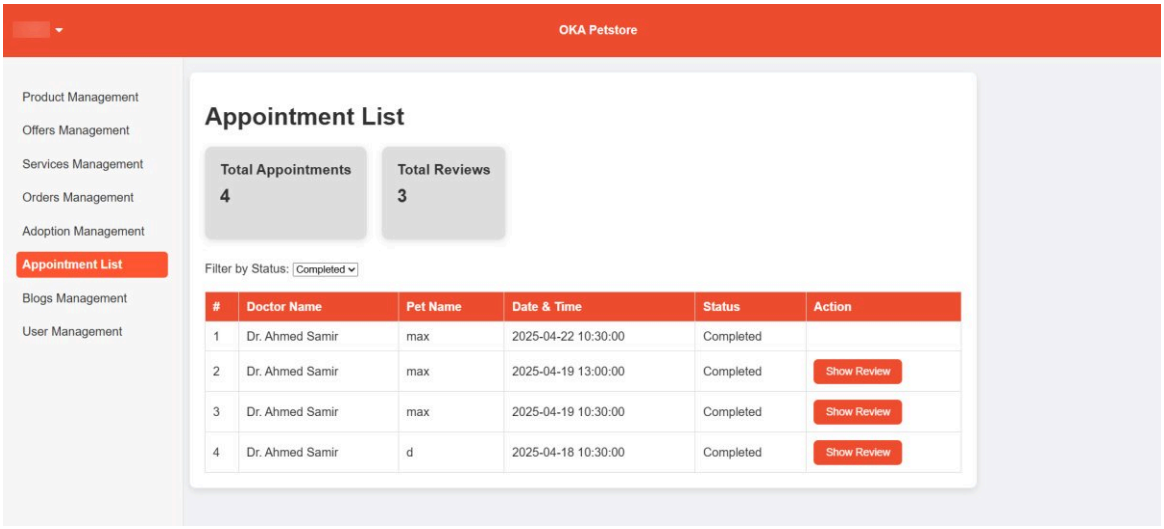


Figure 36 appointment list management

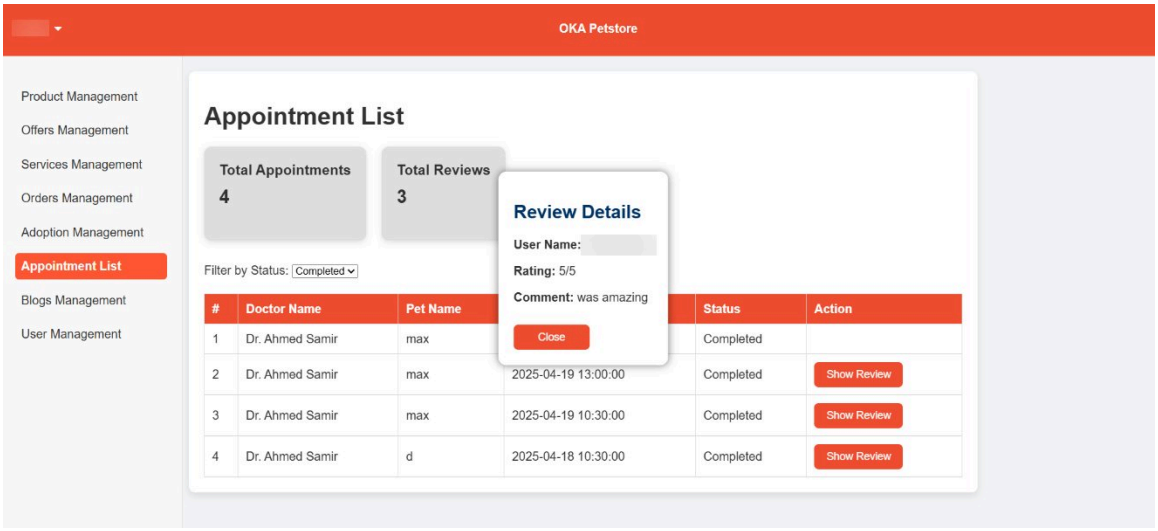


Figure 37 appointment list management

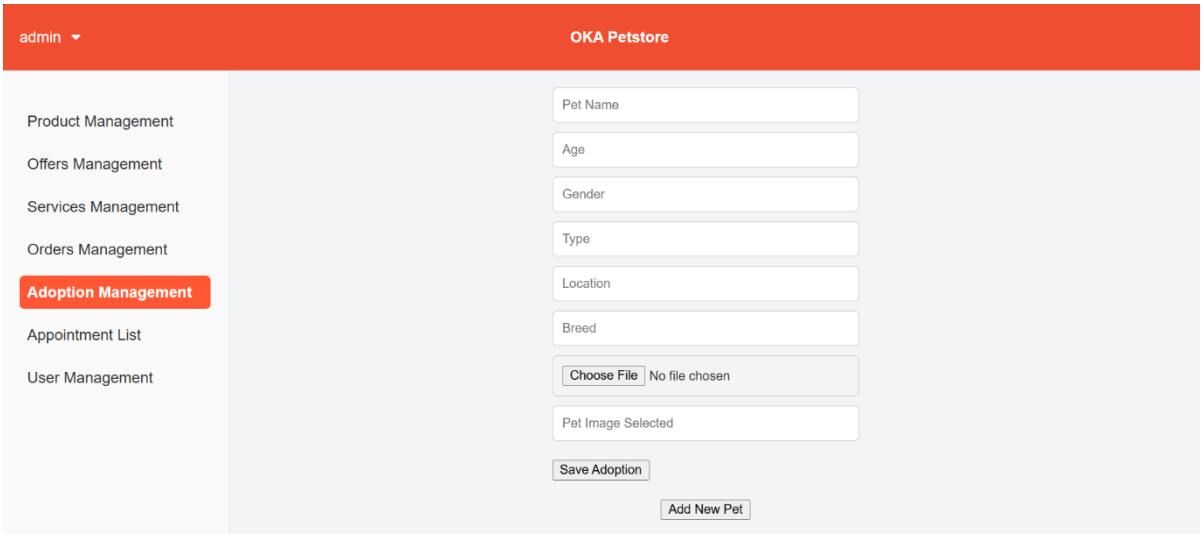


Figure 38 Adoption Management

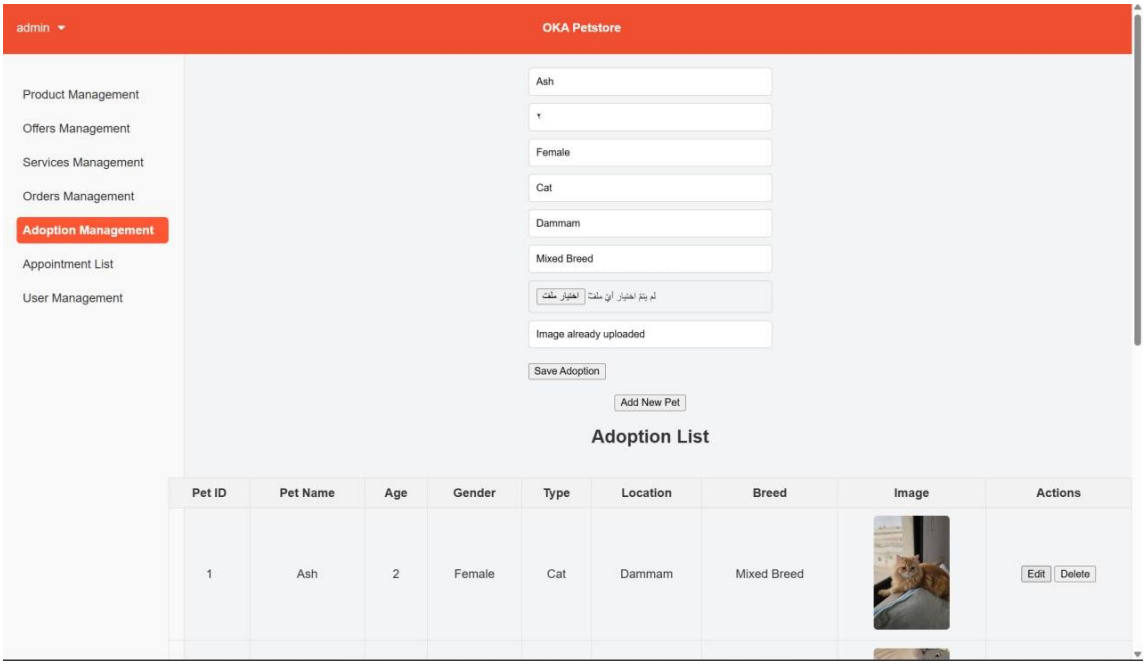


Figure 39 Adoption Management Edit Button

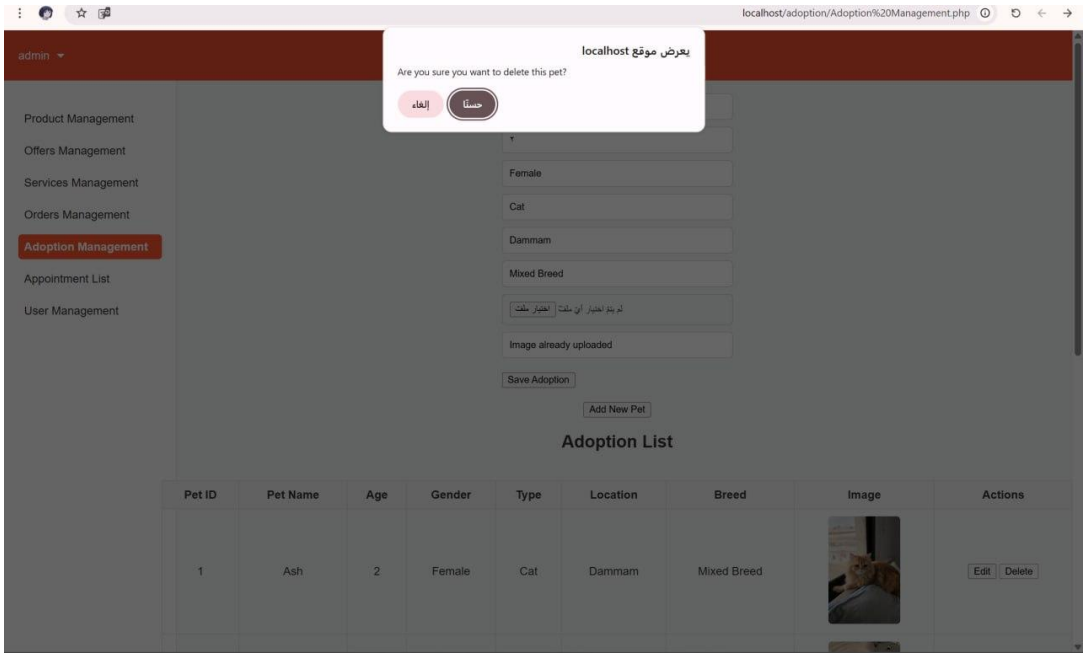


Figure 40 Adoption Management Delete Button

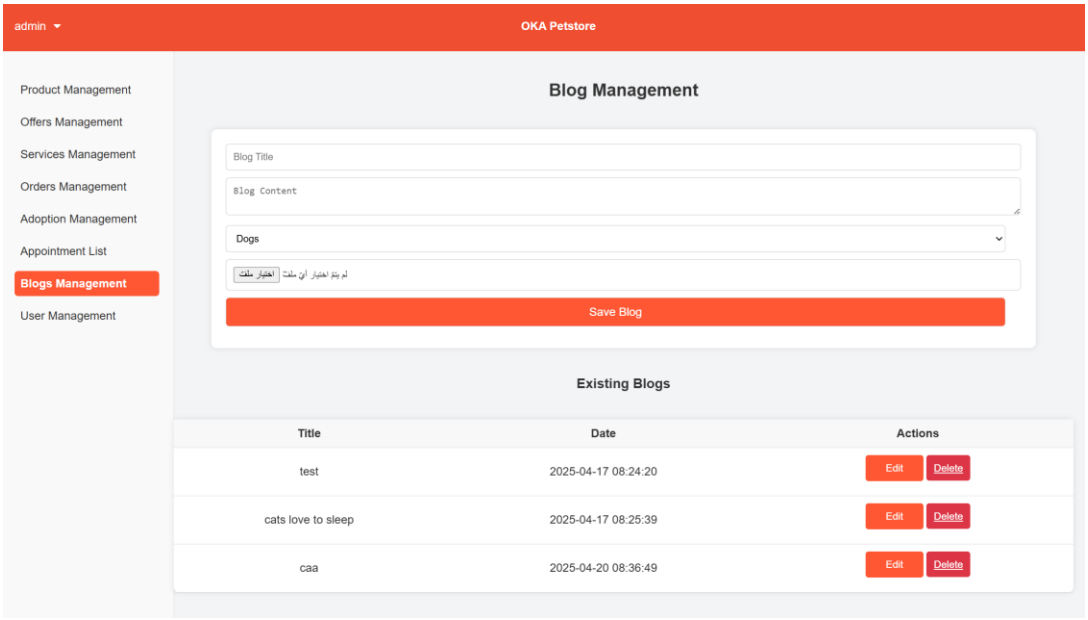


Figure 41 Blogs Management

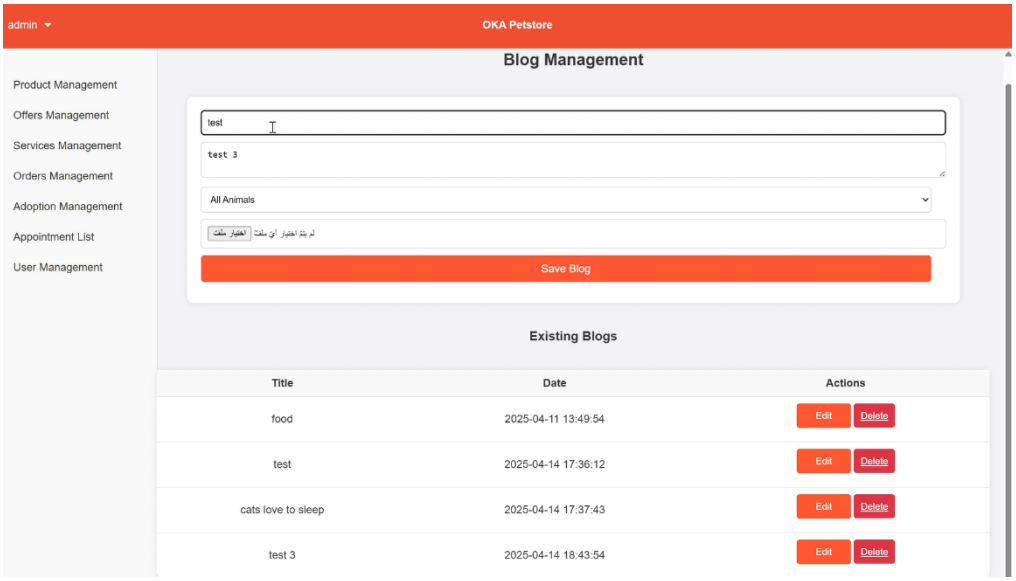


Figure 42 edit button at blogs management

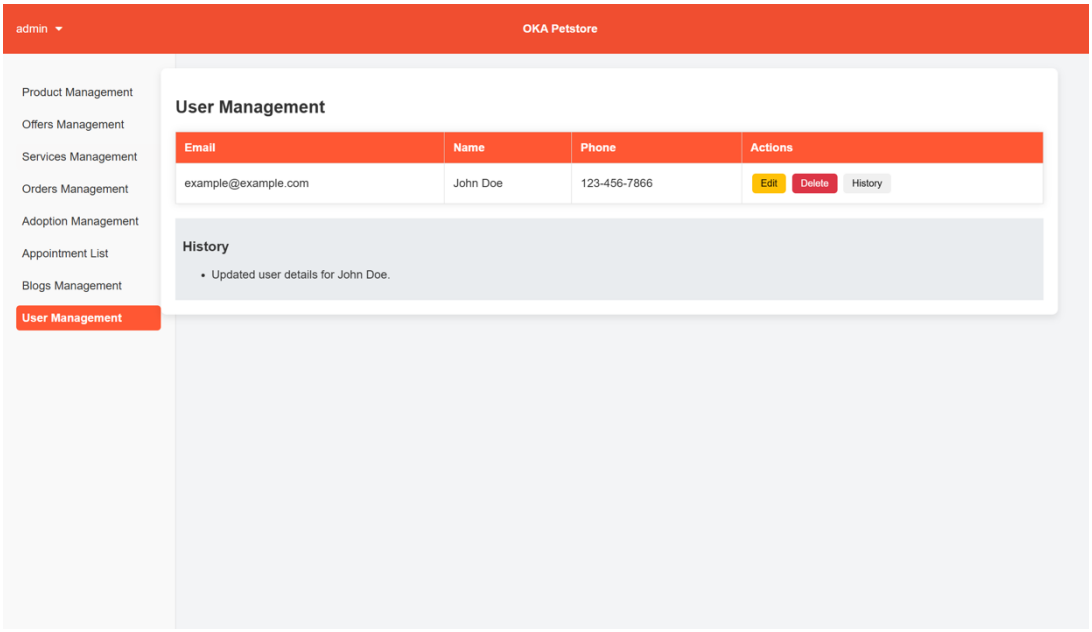


Figure 43 User Management

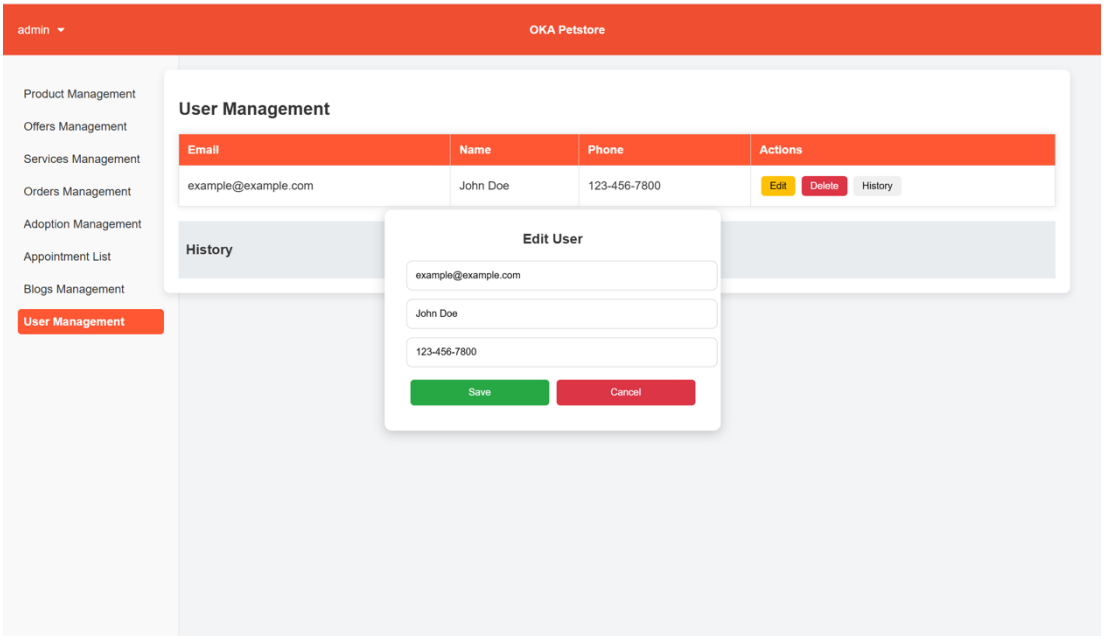


Figure 44 edit pop window

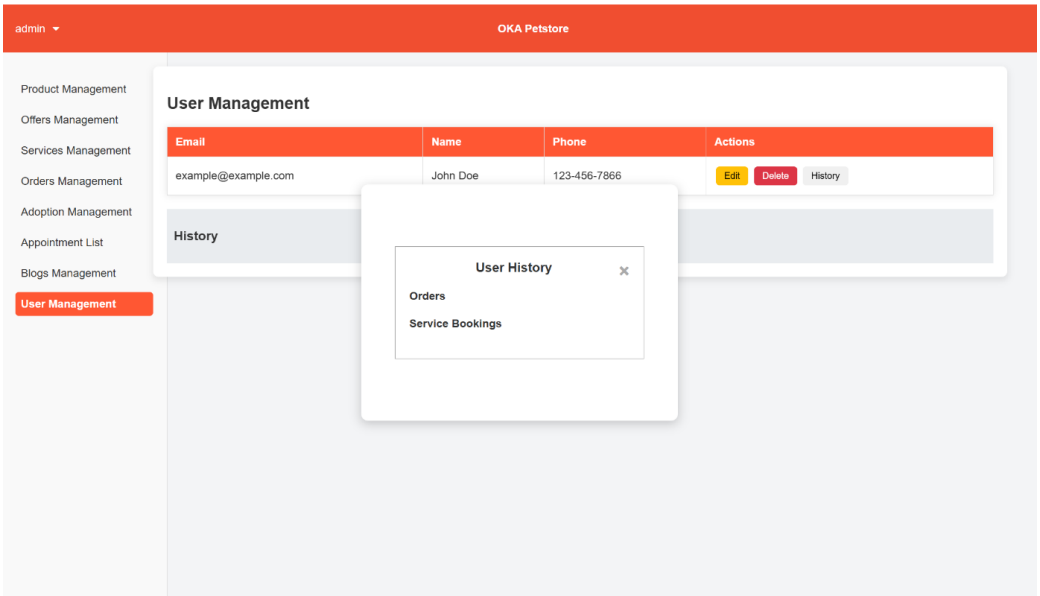


Figure 45 user history

4.5. Super admin interfaces :

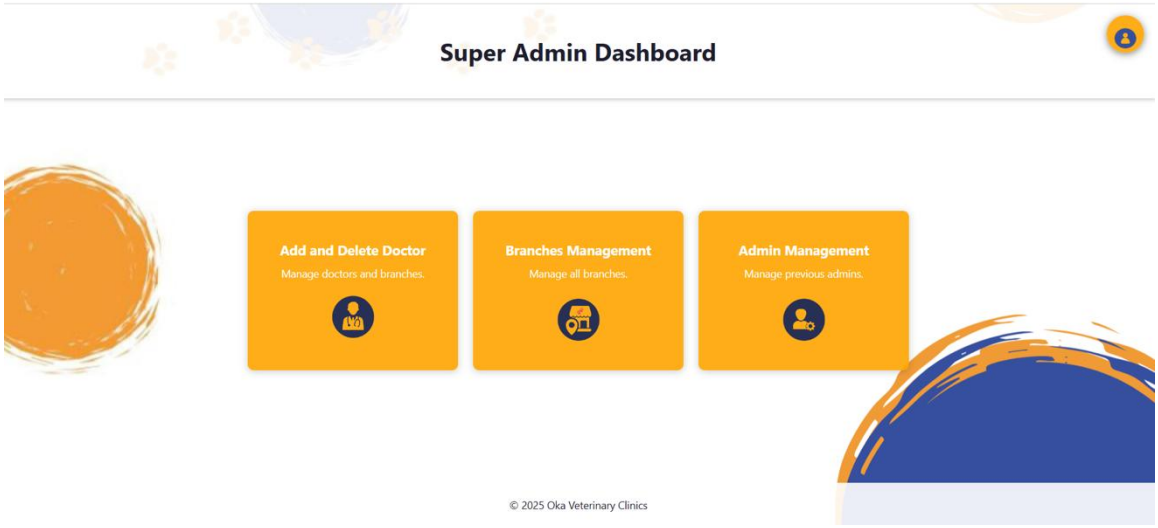


Figure 46 super admin homepage

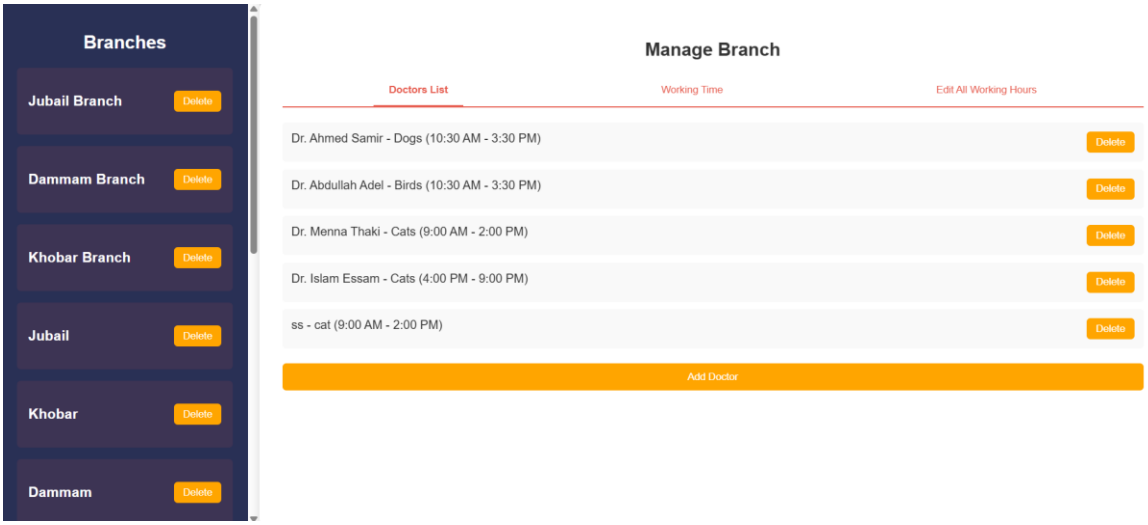


Figure 47 branch management

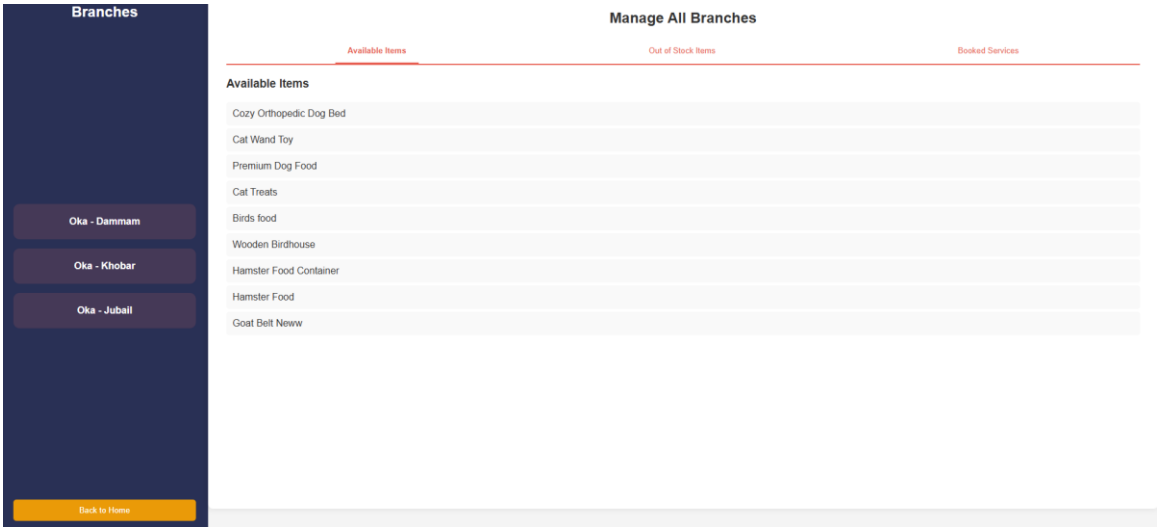


Figure 48 branch management 2

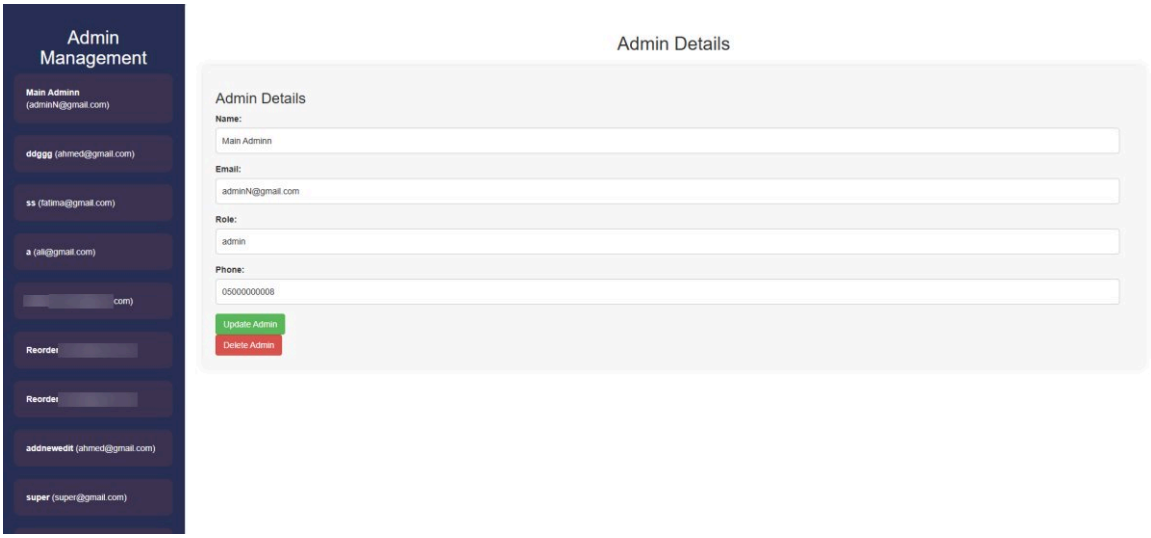


Figure 49 admin management

5. API and cookies :

5.3. Remember Me Feature using Cookies :

In this project, I implemented a secure **"Remember Me"** feature using PHP cookies. When the user logs in and checks the **"Remember Me"** option, a unique token is generated, hashed, and stored in the database. The plain token along with the user ID is stored as cookies in the user's browser for 30 days. On future visits, if the session is not active but the cookies are present, the system validates the token from the cookie against the one stored in the database to restore the user's session.

The implementation helps in maintaining login state across sessions without requiring the user to re-enter their credentials, unless they manually log out.

Additionally, when the user logs out, the cookies (remember_id and remember_token) are deleted and the hashed token in the database is also cleared to prevent unauthorized reuse.

5.3.1.1. Sample Code Snippet:

```
if (!isset($_SESSION['id']) && isset($_COOKIE['remember_id']) && isset($_COOKIE['remember_token'])) {
    $user_id = $_COOKIE['remember_id'];
    $token = $_COOKIE['remember_token'];

    $stmt = $conn->prepare("SELECT user_id, name, profile_img, remember_token FROM user WHERE user_id = ?");
    $stmt->bind_param("i", $user_id);
    $stmt->execute();
    $stmt->store_result();

    if ($stmt->num_rows === 1) {
        $stmt->bind_result($uid, $name, $profile_img, $hashedToken);
        $stmt->fetch();
        if (password_verify($token, $hashedToken)) {
            $_SESSION['id'] = $uid;
            $_SESSION['name'] = $name;
            $_SESSION['profile_pic'] = $profile_img ?: 'profile.png';
        }
    }
    $stmt->close();
}
```

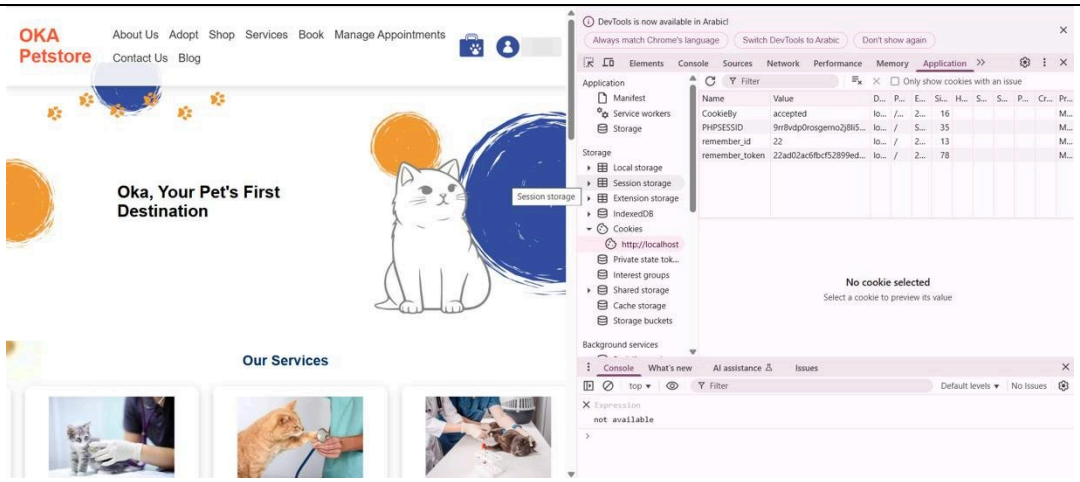


Figure 50 Remember me cookies

remember_token	receive_advertising	profile_img	password	email	phone	name	user_id
KO4M1zJ.UUXJFwjq90F	1	profile.png	...$2y$10$YZOx5YRNIT5dfXs.KH7EXedY106WfmiZ37HLcZG4rCz		123456745		22

Figure 51 database example of using cookies

remember_token	receive_advertising	profile_img	password	email	phone	name	user_id
NULL	1	profile.png	...$2y$10$YZOx5YRNIT5dfXs.KH7EXedY106WfmiZ37HLcZG4rCz		123456745		22

Figure 52 database when user sing out

5.4. Google Calendar Integration using Event Links:

In this project, I implemented an integration with Google Calendar to help users easily remember their appointments. Instead of using complex API authentication (OAuth), I used Google's public Calendar **event link system**, which allows users to create pre-filled calendar events by simply opening a specially formatted URL.

When the user selects a date, time, and branch for a service, they can click **"Add to Google Calendar"** to open a new tab with all their appointment information filled in. The script dynamically builds the URL with the selected details and validates them before allowing access.

To ensure correct usage, the system includes **client-side validation**: if the user attempts to add an event without selecting a valid future date and time, an error message is shown and the link is blocked.

Additionally, the time duration of the calendar event is automatically set to 30 minutes for consistency.

```
function handleCalendar(btn) {
  const card = btn.closest('.service-card');
  const date = card.querySelector('input[name="booking_date"]').value;
  const time = card.querySelector('select[name="booking_time"]').value;
  const branch = card.querySelector('select[name="branch_id"]');
  const serviceName = btn.getAttribute('data-service');
  const branchText = branch?.selectedOptions[0]?.text || 'OKA Clinic';
  if (!date || !time) {
    alert("❌ Please select both date and time before adding to calendar.");
    return;  }
  const selectedDate = new Date(date);
  const today = new Date();
  today.setHours(0, 0, 0, 0);

  if (selectedDate < today) {
    alert("⚠️ The selected date is in the past. Please choose a valid future date.");
    return;  }
  const startDateTime = new Date(`${date} ${time}`);
  const endDateTime = new Date(startDateTime.getTime() + 30 * 60000);
  function formatDate(dt) {
    return dt.toISOString().replace(/[-:]/g, '').split('.')[0] + 'Z';  }

  const url = `https://calendar.google.com/calendar/render?action=TEMPLATE`
    + `&text=${encodeURIComponent(serviceName)}`
    + `&details=${encodeURIComponent("Appointment at OKA Petstore Clinic")}`
    + `&location=${encodeURIComponent(branchText)}`
    + `&dates=${formatDate(startDateTime)}/${formatDate(endDateTime)}`;

  window.open(url, '_blank');
}
```

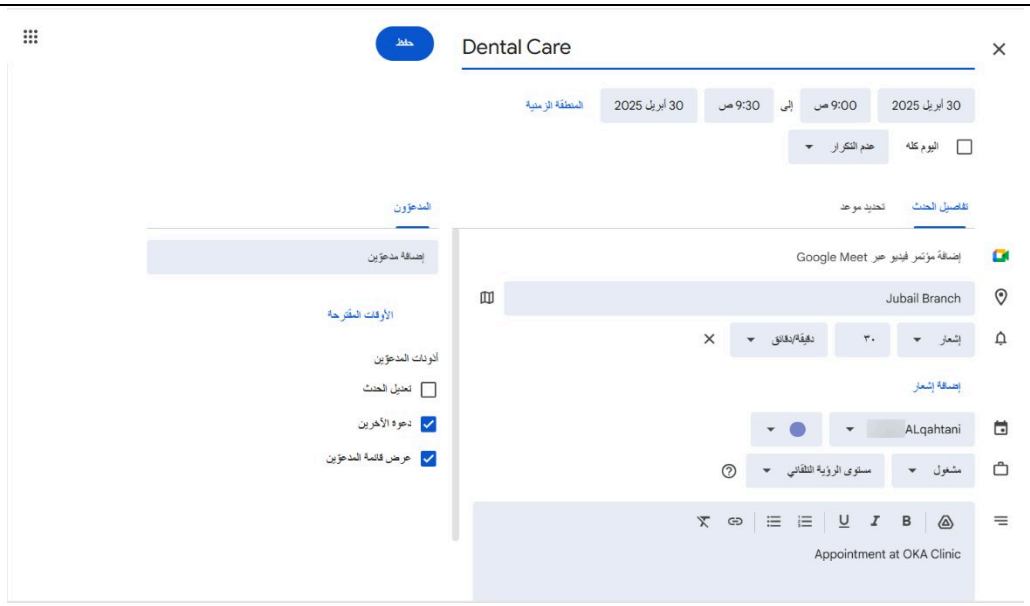


Figure 53 Google Calendar Integration using Event Links

6. Fulfilled requirements

a- Successfully Implemented Requirements

The following requirements were fully implemented and verified:

- User Registration and Account Management
    - A secure registration and login system was implemented with validation for email, phone number, and password strength.
    - "Remember Me" functionality allows users to maintain login sessions using secure cookies and hashed tokens.
    - Users can update profile data and view personal dashboards upon signing in.
    - Attempting to create an account with a previously registered email or phone is correctly blocked.
- Access Control Based on User Status
    - Booking appointments or shopping for products is strictly restricted to registered and logged-in users.
    - In contrast, adoption-related activities (viewing available pets, submitting adoption applications, or posting a rehoming request) are accessible without registration, encouraging public participation in animal welfare.
- Pet Profile and Appointment Management
    - Logged-in users can add, edit, or remove pet profiles.
    - Users may not delete pets that are linked to existing appointments.
    - Users can book appointments with selected doctors for their pets, with prevention of duplicate or past-date bookings.
    - Appointment status updates (e.g., Cancelled, Completed) are fully functional.
- Service Booking and Calendar Integration
    - Users can book services such as grooming and check-ups with validated date and time selections.
    - The system includes a dynamic Google Calendar event link to help users remember appointments.

- Booking past dates or submitting incomplete forms is appropriately blocked.

- Online Store and Cart System
  - Logged-in users can browse products, add items to their cart, modify quantities, and remove items.
  - Attempting to add or purchase out-of-stock items is prevented, with immediate user feedback.
  - Toast messages provide clear confirmations for actions like order placement or an empty cart.
- Adoption and Pet Rehoming (Publicly Accessible)
  - Any user—registered or not—can view adoptable animals and submit adoption applications.
  - Users can also submit a request to rehome their pets through a validated form.
  - Admins manage adoption listings and can approve or remove submissions.
- Order History and Review System
  - Registered users can view previous orders and submit ratings and reviews for completed appointments.
- Admin and Super Admin Functionalities
  - Admins have access to product management, user accounts, offers, orders, services, blogs, and adoption forms.
  - Super admins can manage clinic branches and administrative accounts through a dedicated interface.
  - All admin forms include validation checks for input logic (e.g., no negative discounts or past dates).
- Content Management and Blogs
  - Admins can add, update, and delete blog posts with title, content, image, and category fields.
  - Static content such as “About Us” and “Terms of Service” is accessible to all users.
- Error Handling and Feedback
  - The system includes full form validation and context-sensitive error messages.
  - Users are informed of successful and unsuccessful actions through toast notifications and modals.
  - Testing scenarios confirmed appropriate behavior for login failures, invalid bookings, and data entry errors.

- People with special needs integration

b- Partially Implemented or Adjusted Features

- Password Reset via Email
    - The system includes token-based password reset functionality. However, the feature does not send actual confirmation emails due to the absence of a subscription to an email delivery service.
- Payment Integration and Receipt Generation
    - Payment method selection and order finalization are implemented and recorded in the database.
    - Integration with a live payment gateway and automated receipt generation is not completed, as it requires a third-party subscription.
- JavaScript Modal Windows
    - While implementing modal windows for UI feedback, an issue occurred when the JavaScript was moved to an external file. The pop-up did not render properly.
    - This was resolved by placing the JavaScript directly within the same HTML file, restoring expected behavior.

c- Unimplemented Features

- Automated Notifications (Email/SMS)
- Notifications such as appointment reminders were not included, as they require integration with external APIs.

7. Testing cases :

7.3. Sigh up and log in

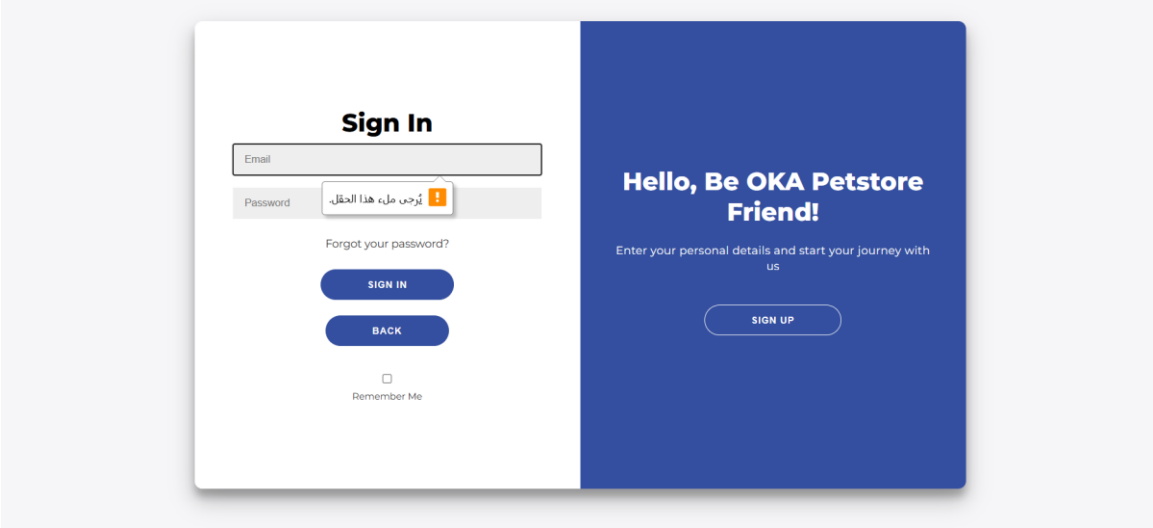


Figure 54 when user try to sign in but didn't full all requirements

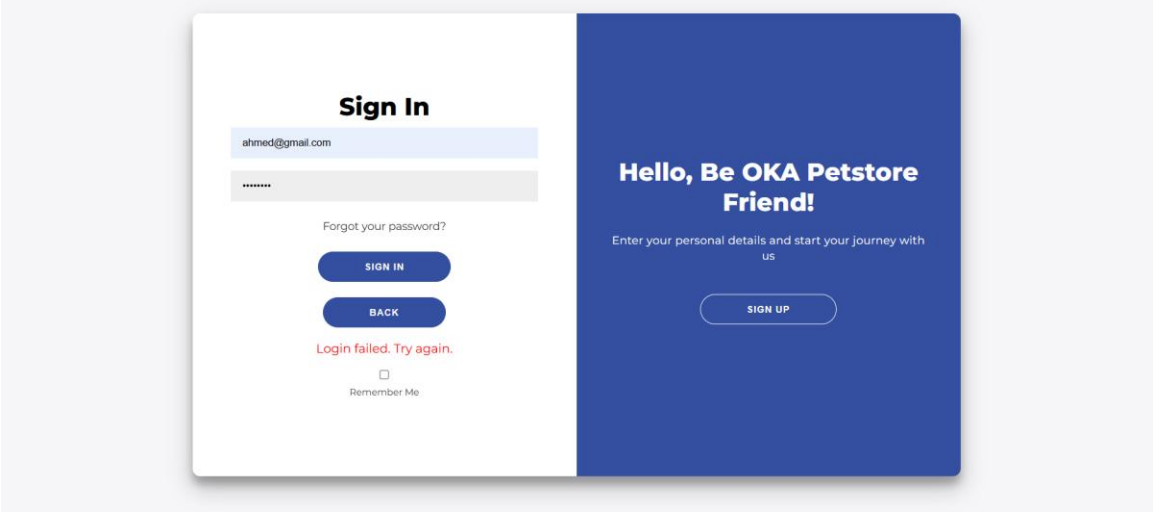


Figure 55 when user enter email not registered in database or server problem

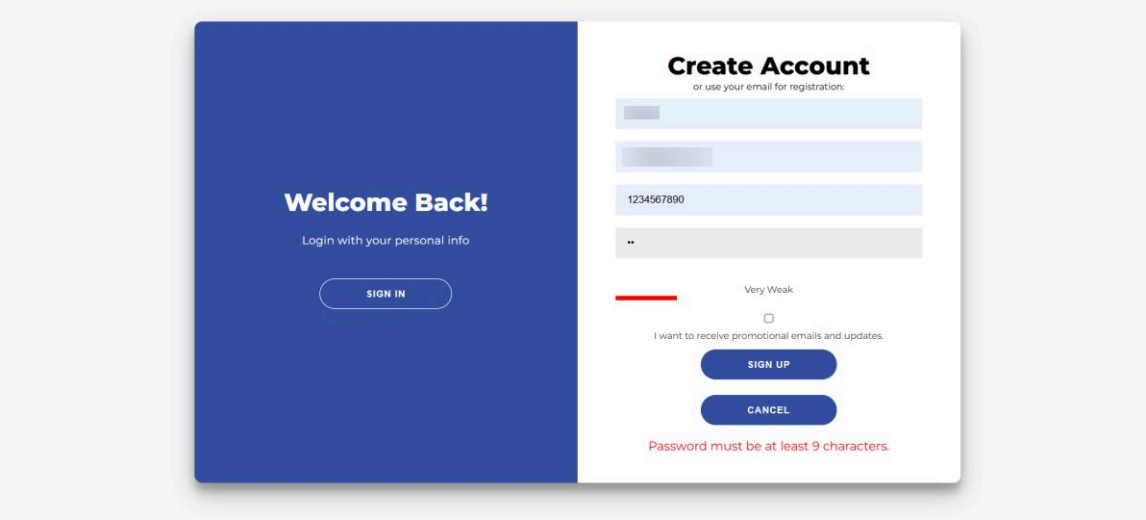


Figure 56 handling all user has at least 9 characters in their password

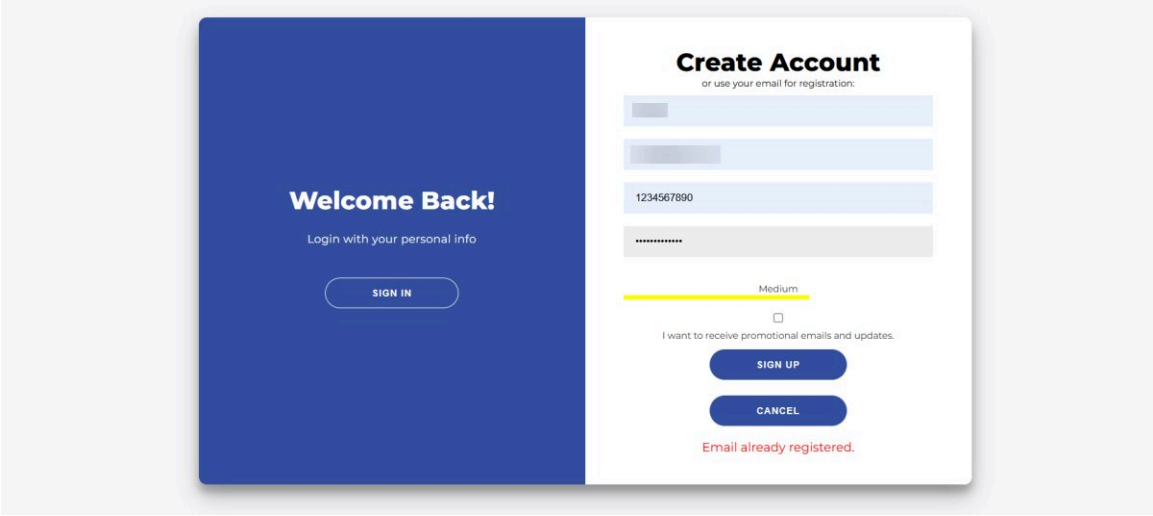


Figure 57 handling duplicated email (no user can create two account with same email or phone number)

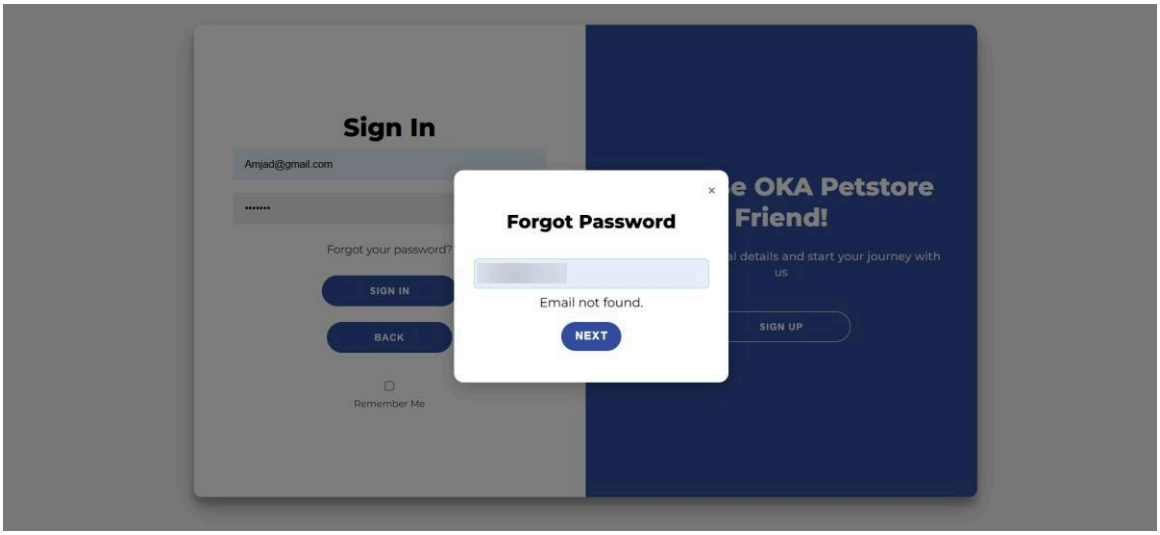


Figure 58 handling rest password only to registered users accounts

7.4. Shop and Cart:

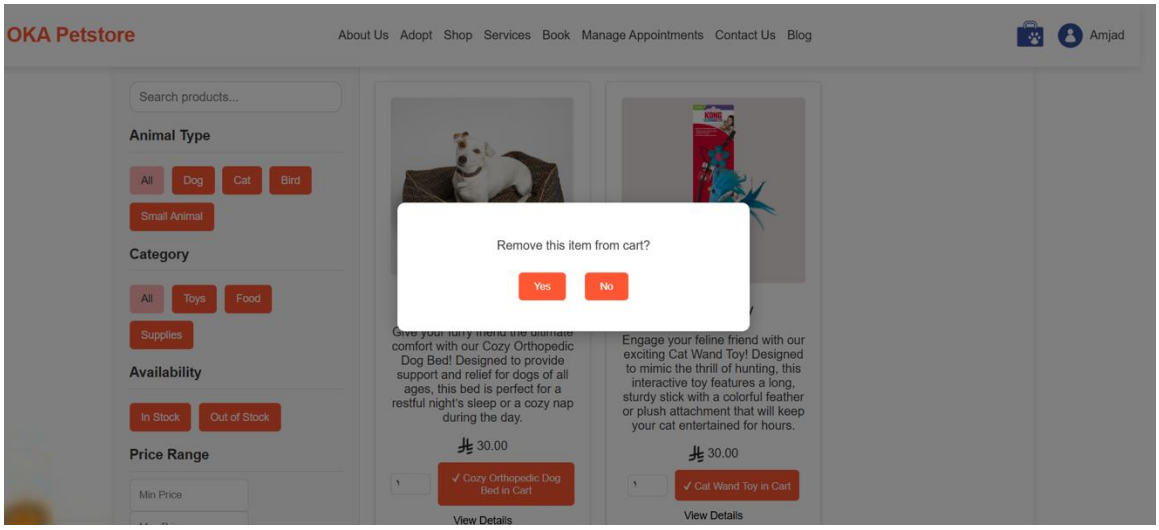


Figure 59 Remove cart iteam

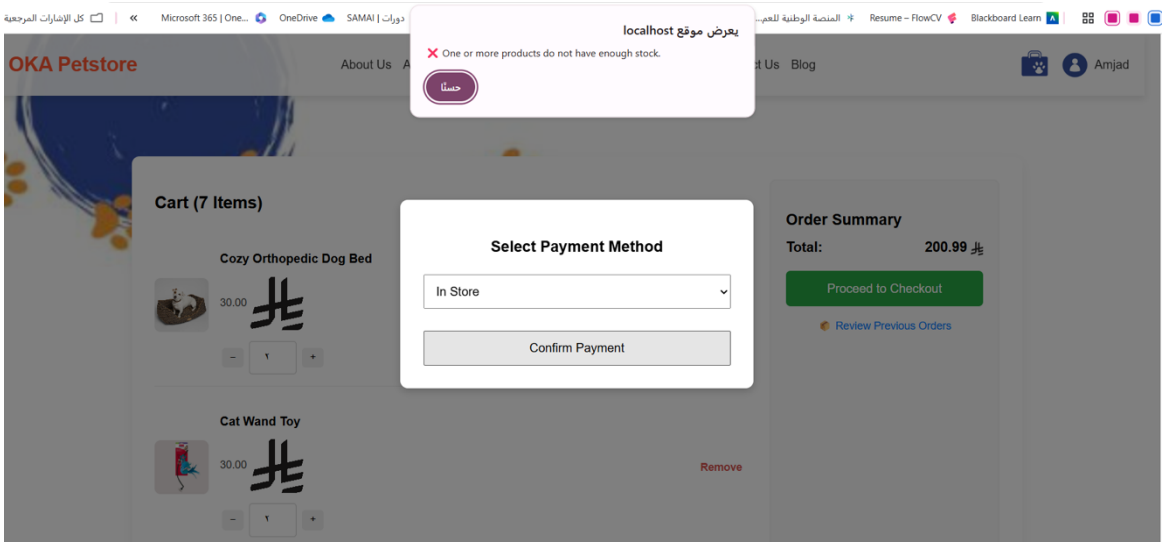


Figure 60 to insure all process are valid users can't confirm order when product is out of stuck

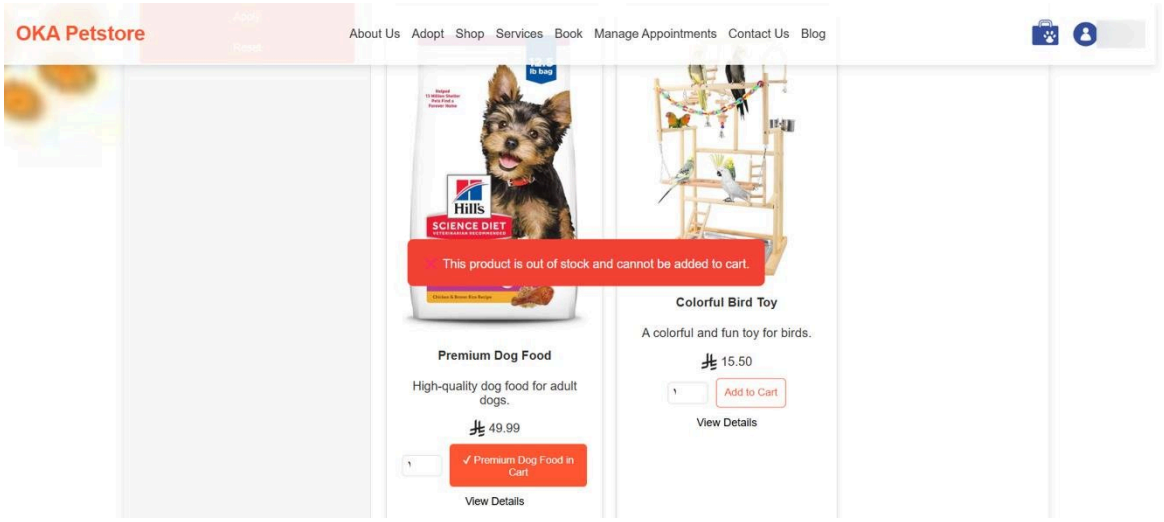


Figure 61 to insure all process are valid user can't add product out of stuck to their cart

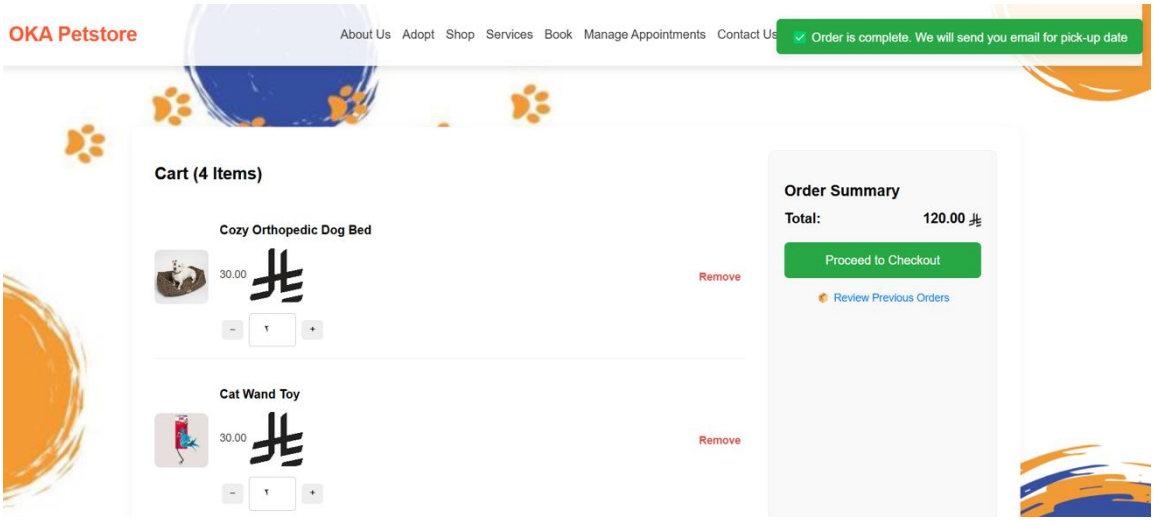


Figure 62 Toast message when confirm order

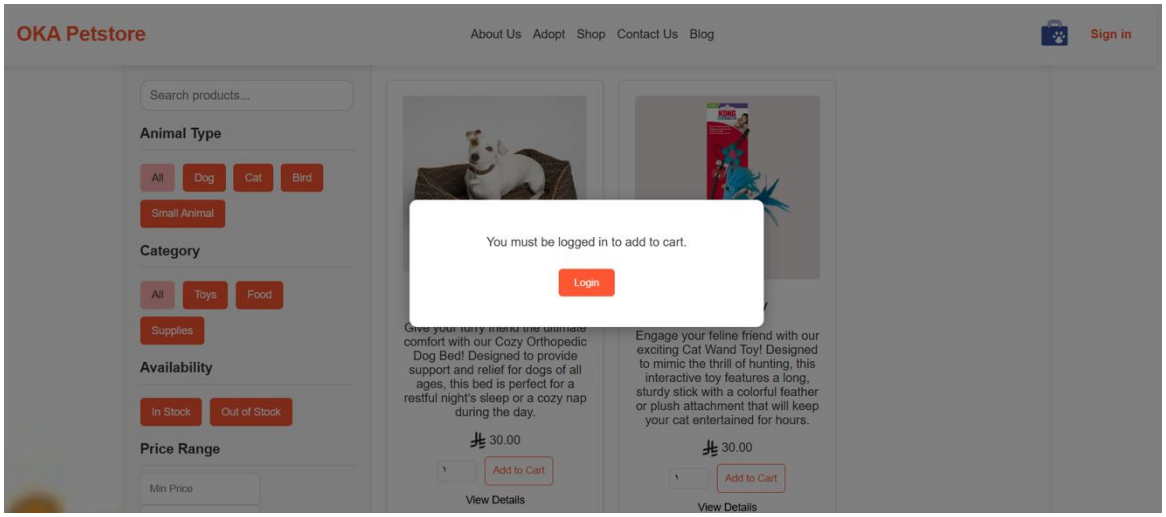


Figure 63 User should log in or sign up before add product to cart

7.5. Offers management

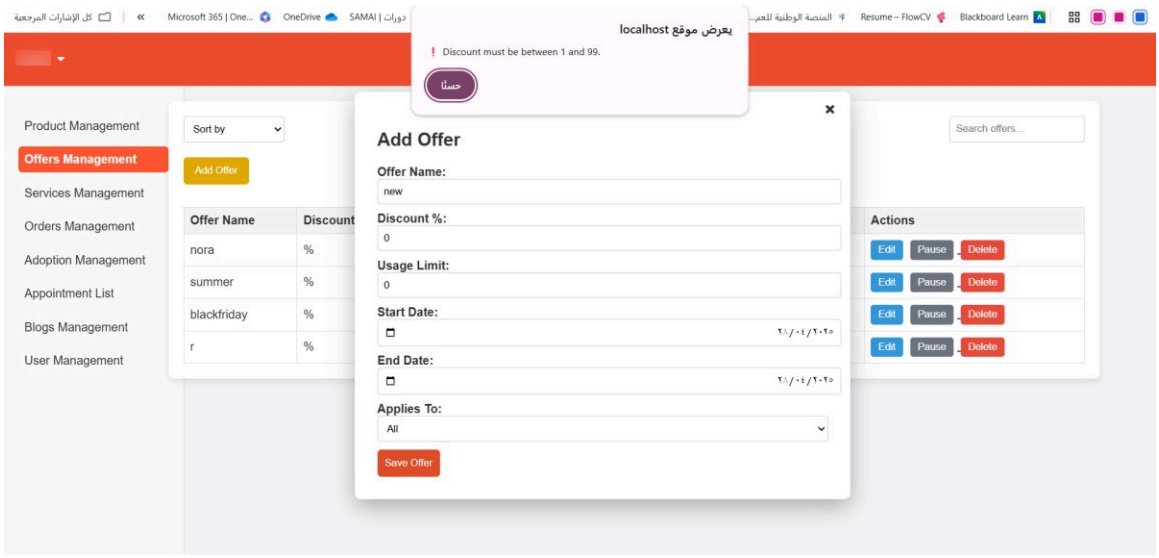


Figure 64 handling logic error when admin want to add offer but discount is less than 0 or equal 0

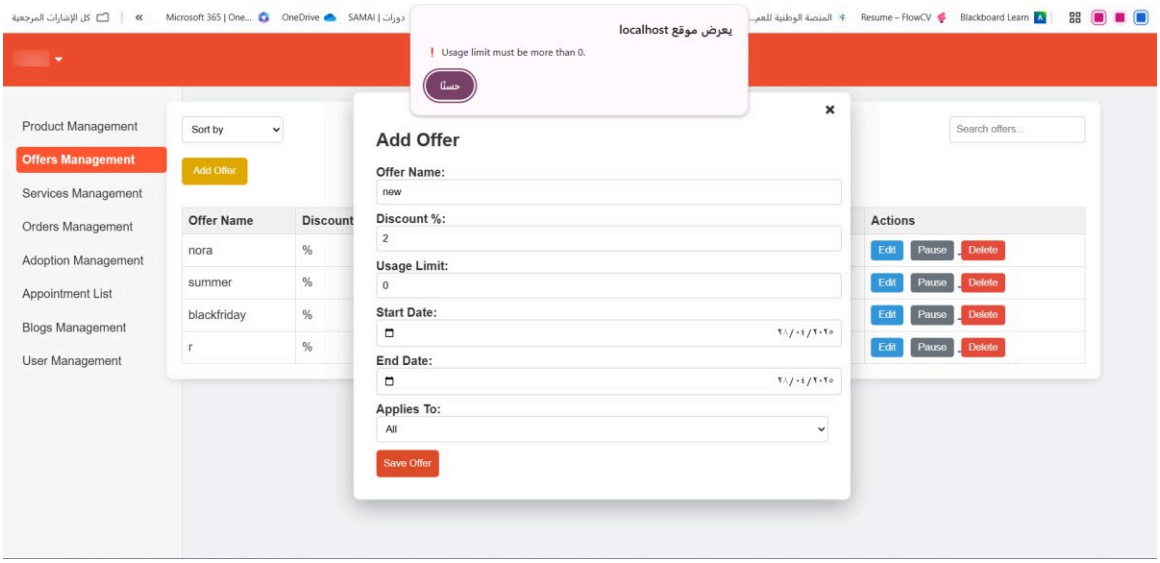


Figure 65 handling logic error when admin want to add offer but Usage limit is less than 0 or equal 0

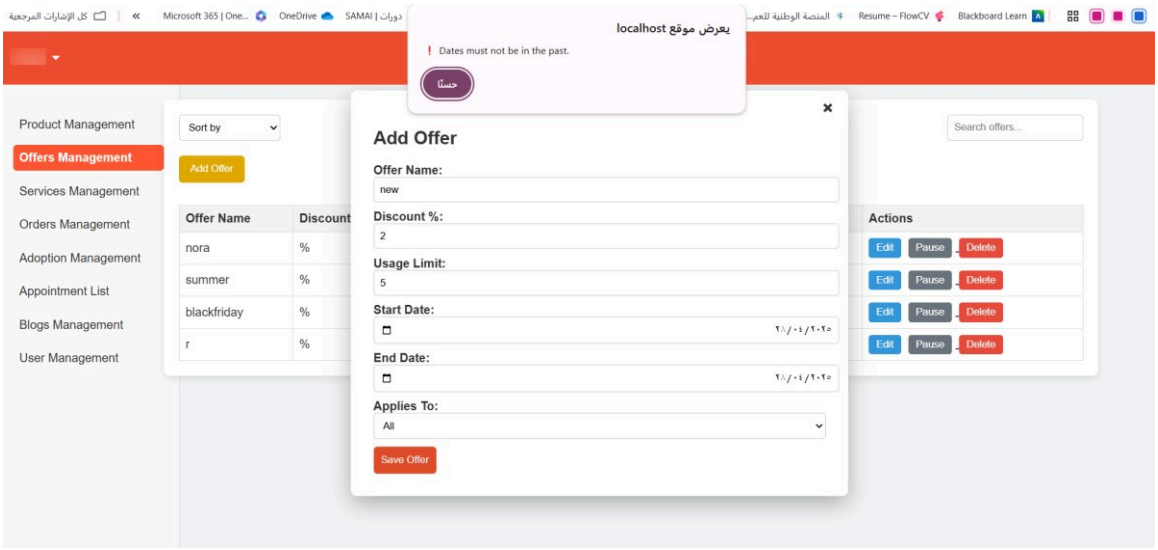


Figure 66 handling logic error when admin want to add offer but date is in past date

7.6. Services

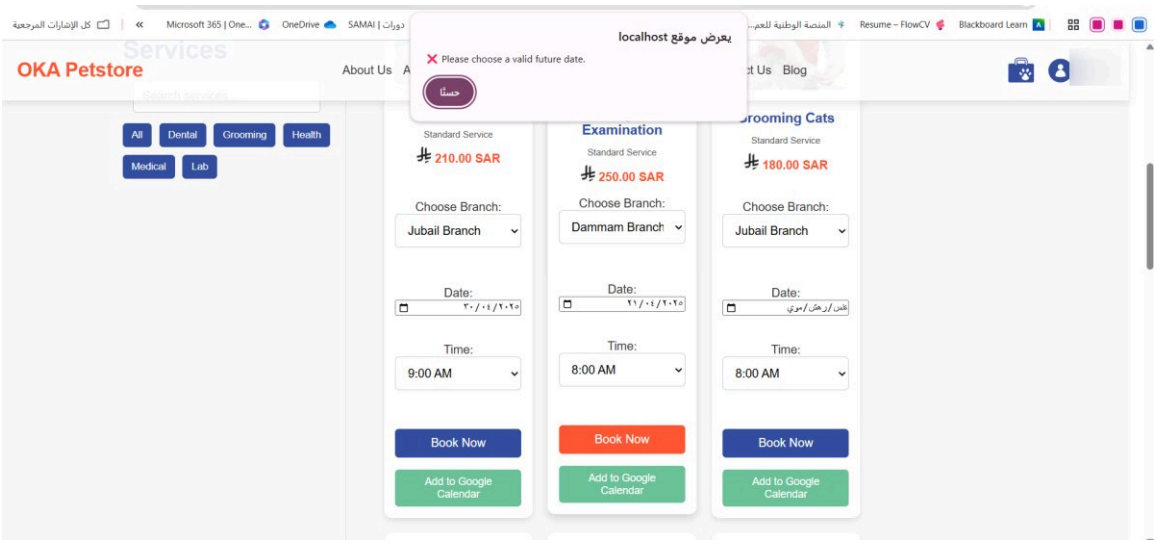


Figure 67 handling logic error when user book serves but date is in past date

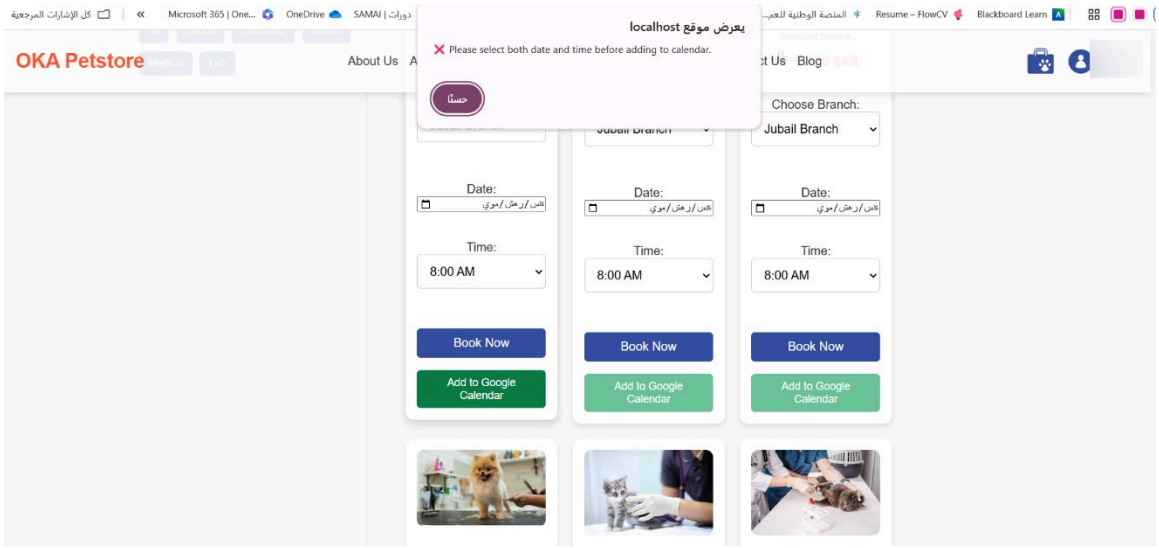


Figure 68 handling logic error when user book serves but didn't full requirements

7.7. Book

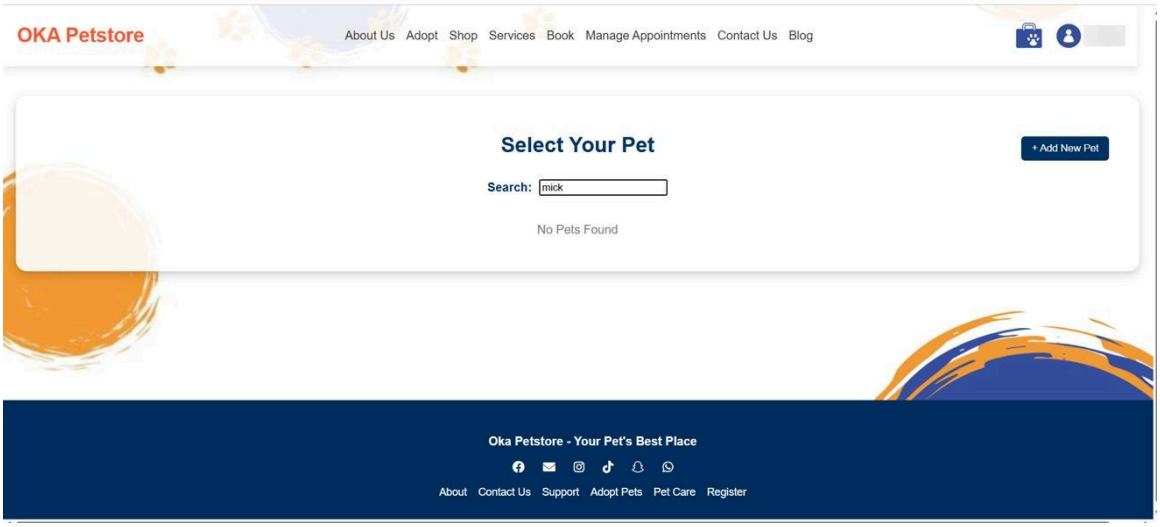


Figure 71 Handling search logic when user types a pet name that does not exist in the database

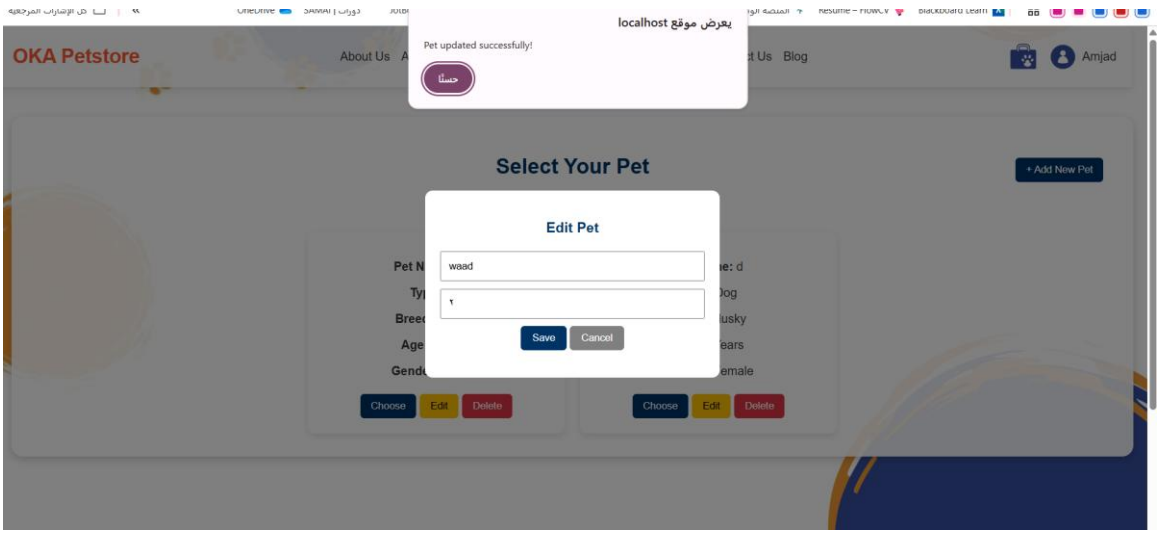


Figure 72 Handling update logic when user edits pet information and clicks save successfully.

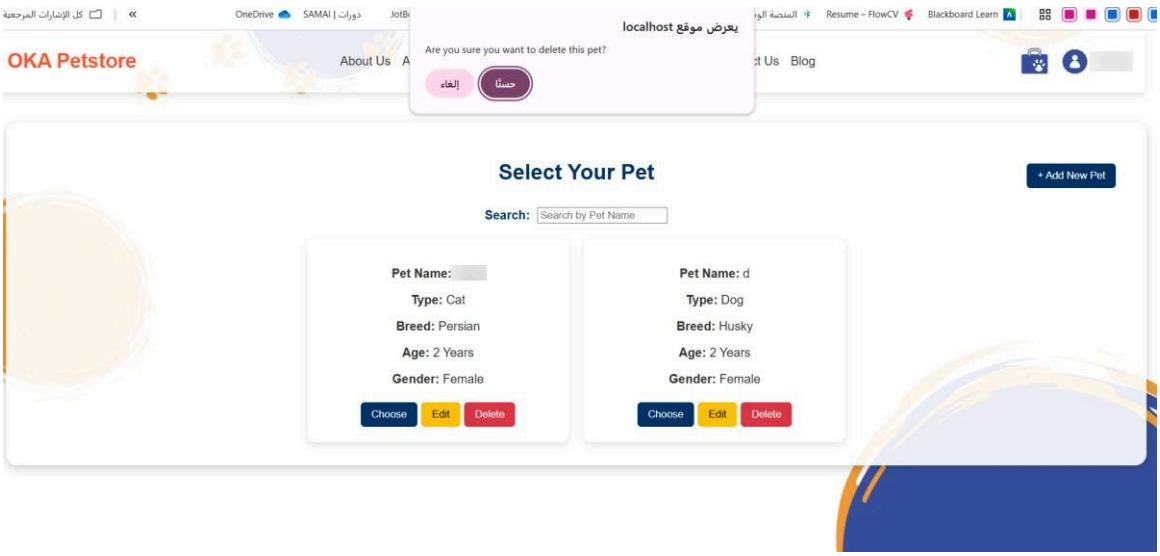


Figure 73 Handling confirmation prompt when user attempts to delete a pet from the selection list.

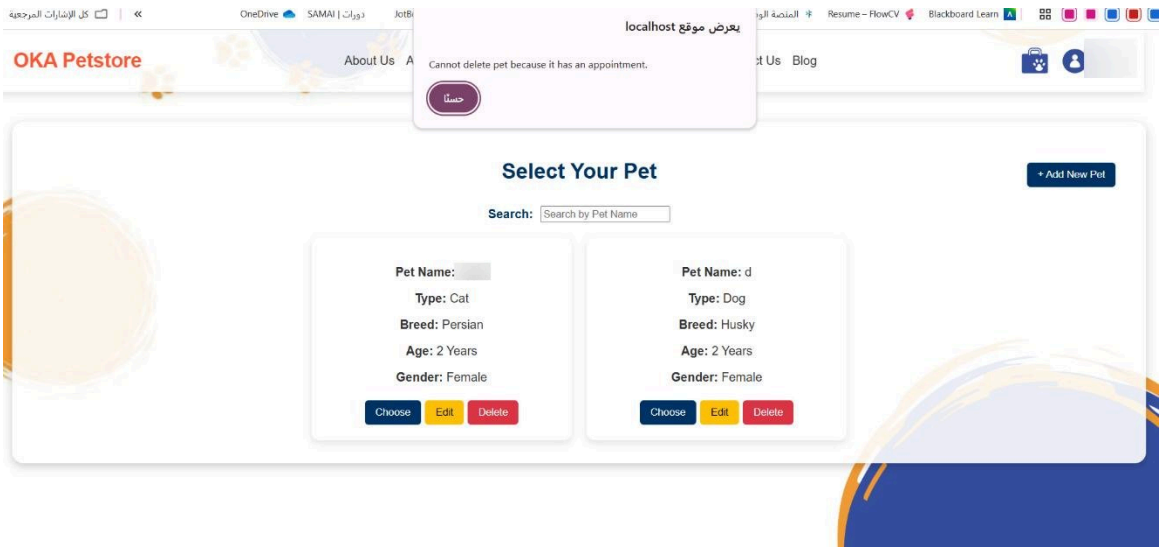


Figure 74 Handling error message when user tries to delete a pet that is linked to an existing appointment.

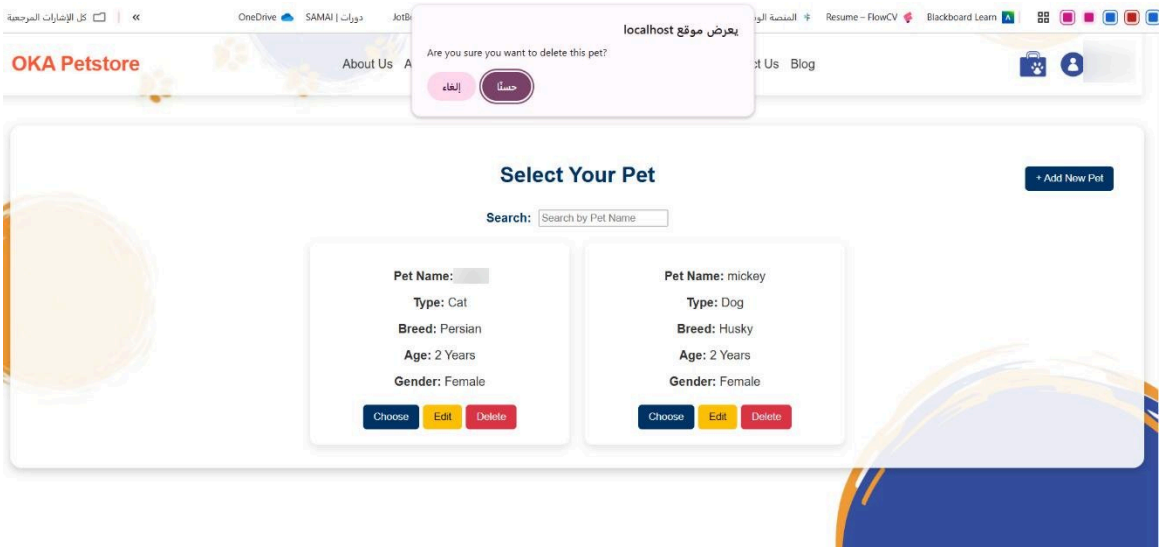
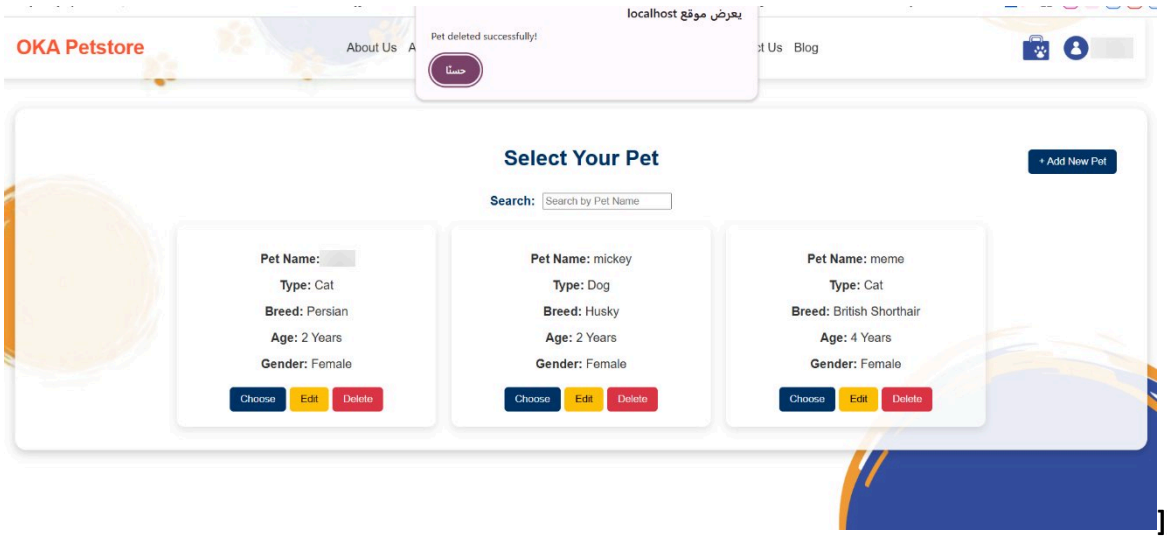


Figure 75 Handling successful deletion confirmation when user attempts to delete a pet with no linked appointments.



]]

Figure 76 Handling success message after deleting a pet that has no existing appointment in the system.

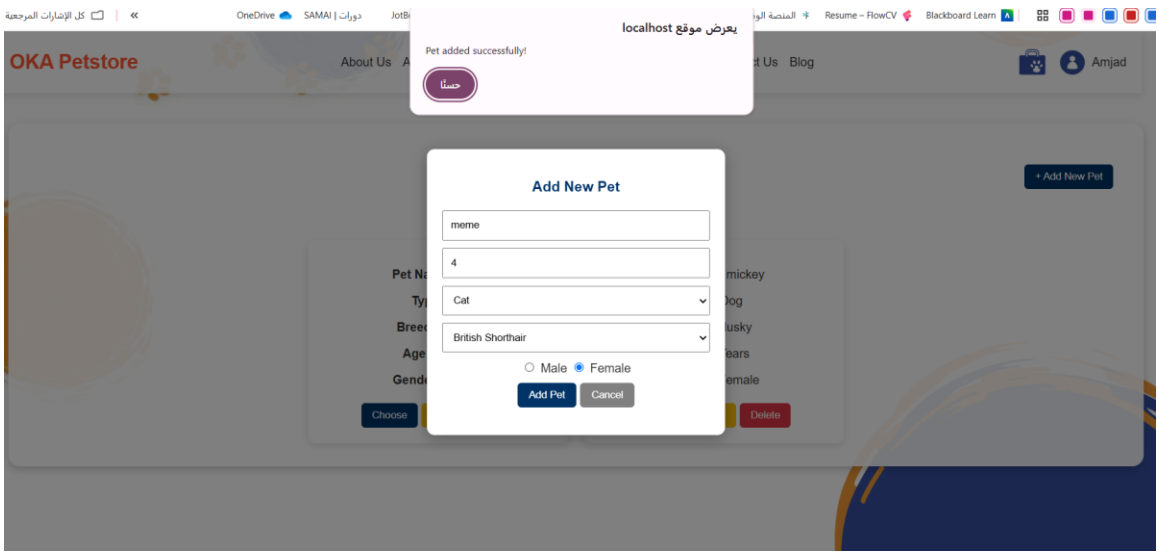


Figure 76 Handling success message when a new pet is added with all required fields filled correctly.

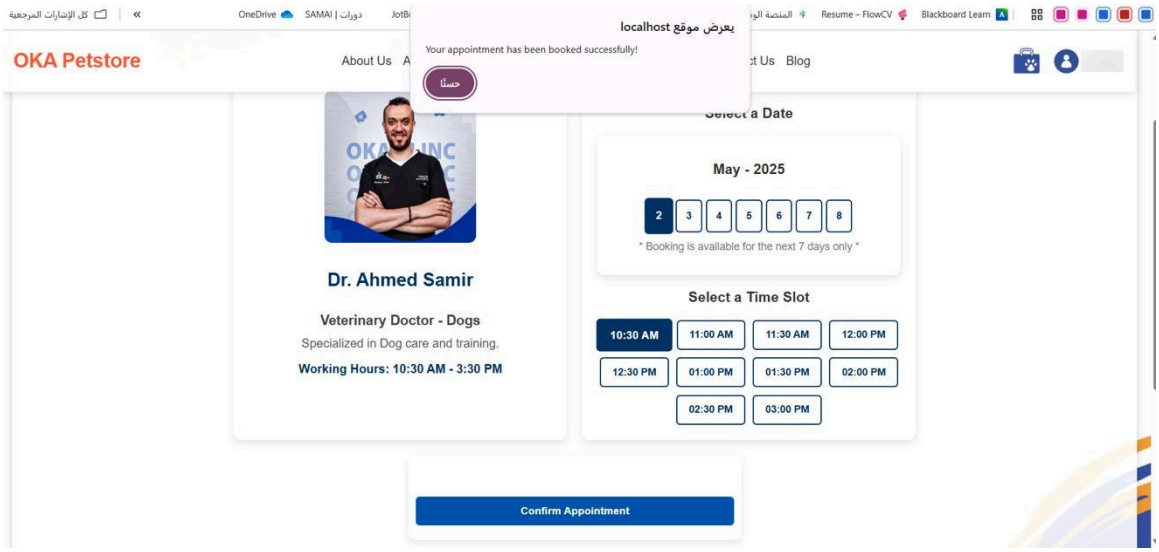


Figure 77 Handling success message after booking an appointment with all required selections completed correctly.

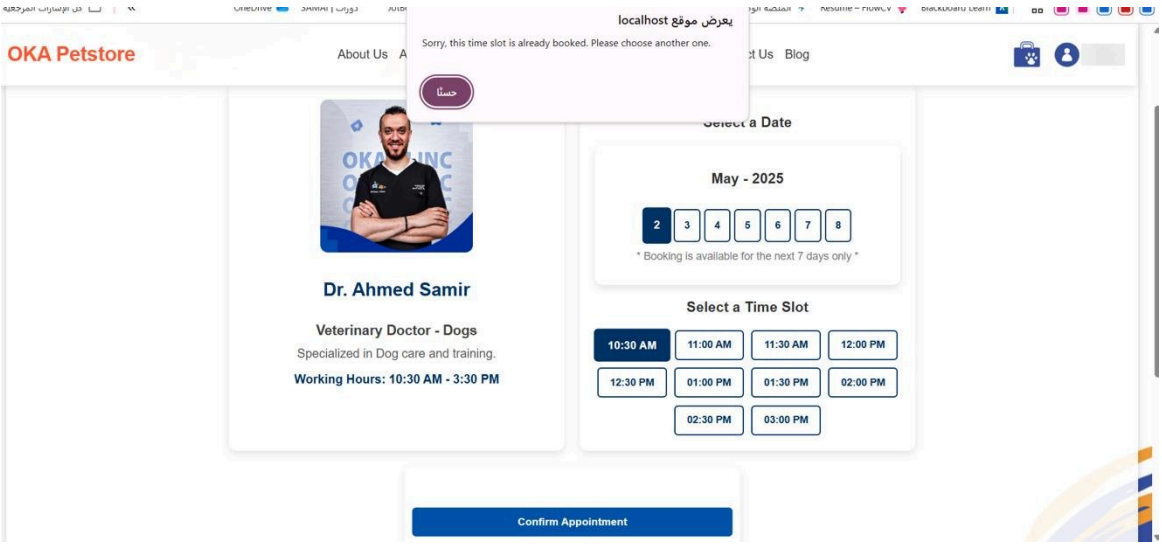


Figure 78 Handling booking conflict error when user selects a time slot that has already been reserved.

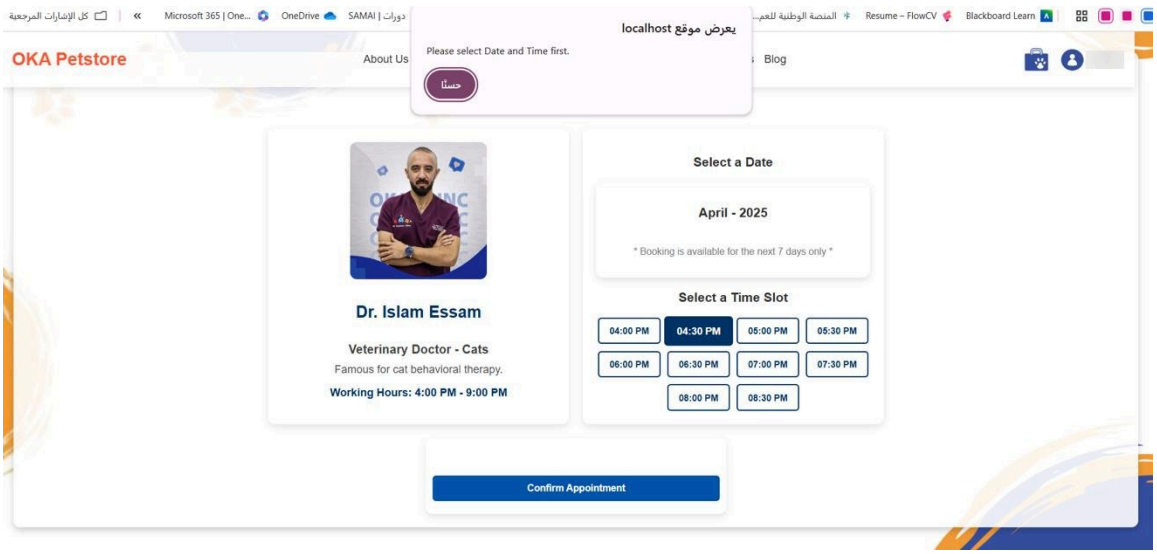
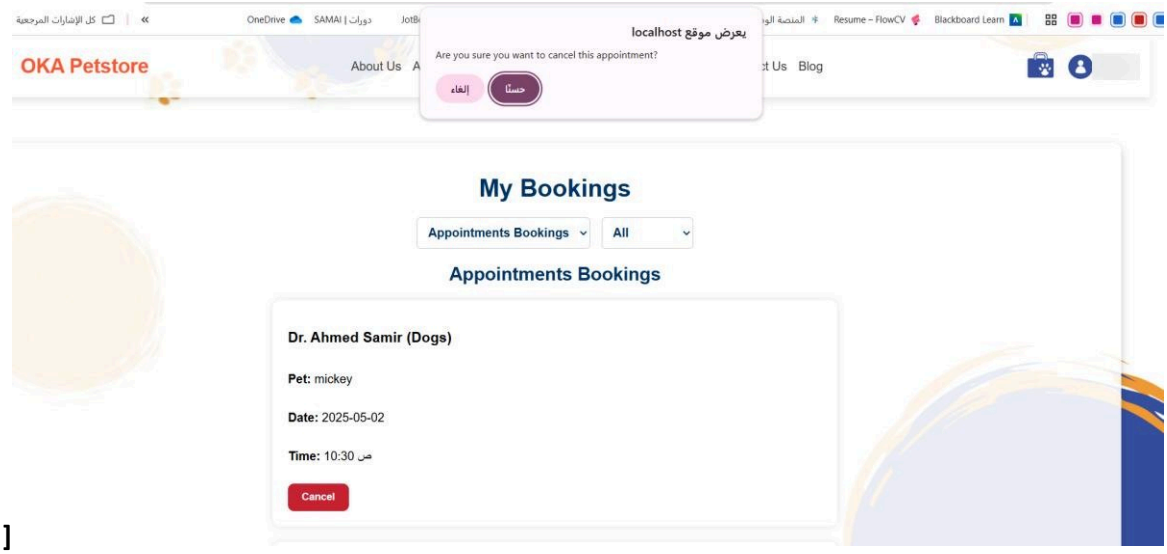


Figure 79 handling logic error when user book but didn't full requirements



]

Figure 80 Handling cancellation confirmation when user attempts to cancel an upcoming appointment from the bookings page.

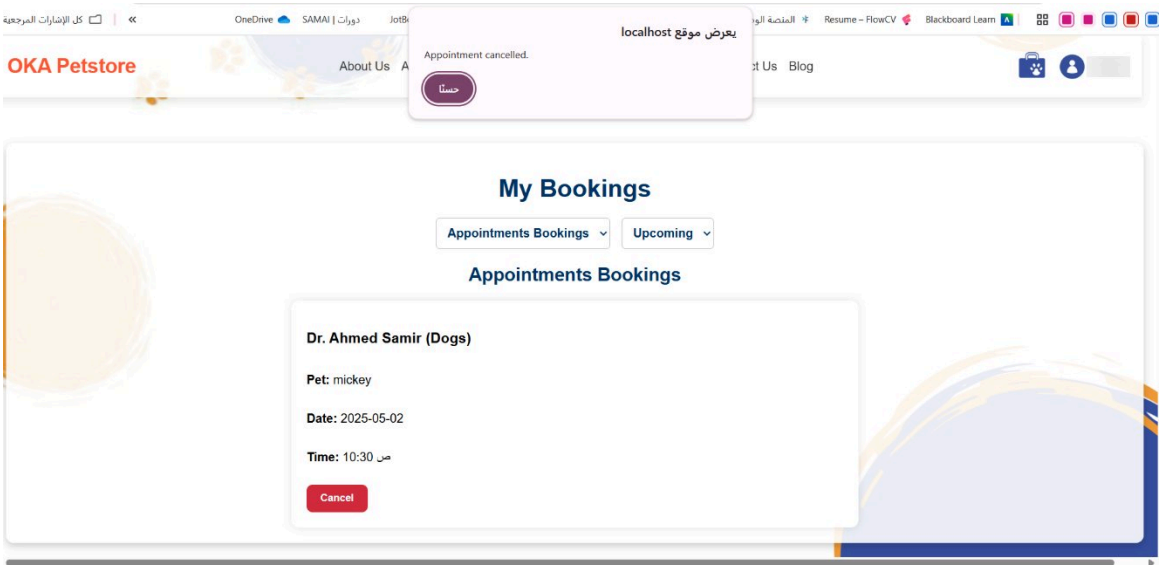


Figure 81 Handling success message after user confirms and completes the cancellation of an upcoming appointment.

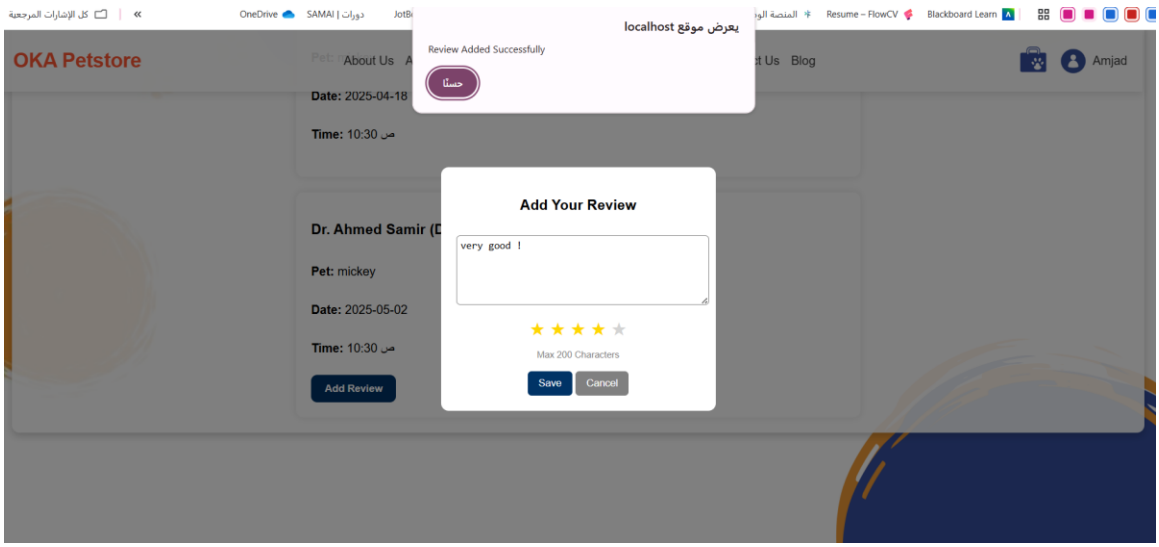


Figure 82 Handling success message when user submits a valid review with text and star rating after completed appointment.

7.8. Blogs Management

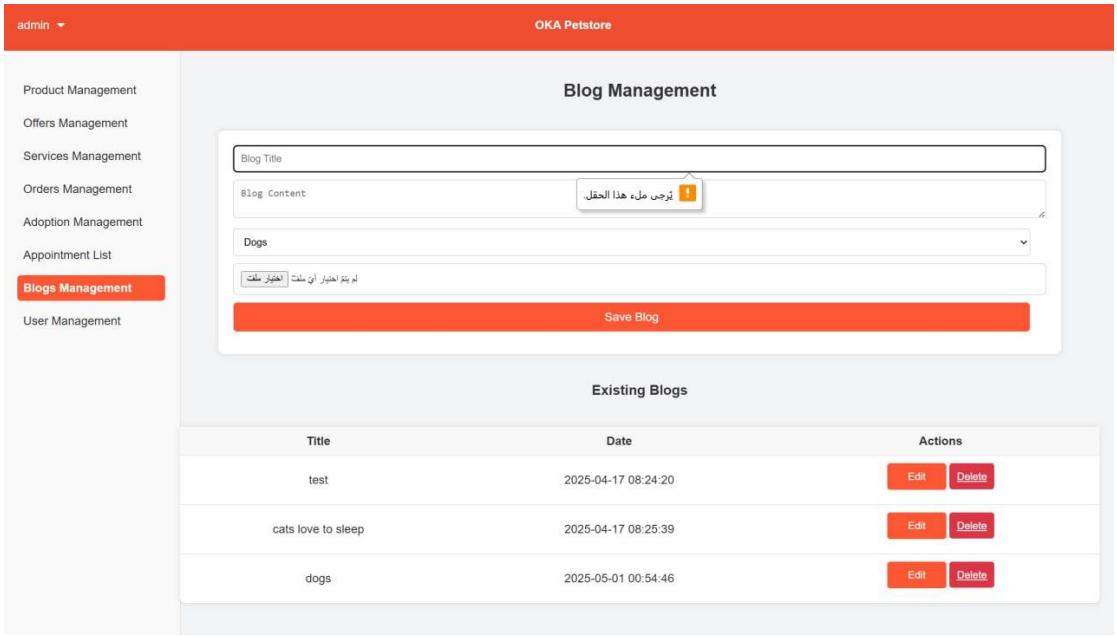


Figure 83 When a manager tries to save a blog without filling all the fields

1.7 Adoption

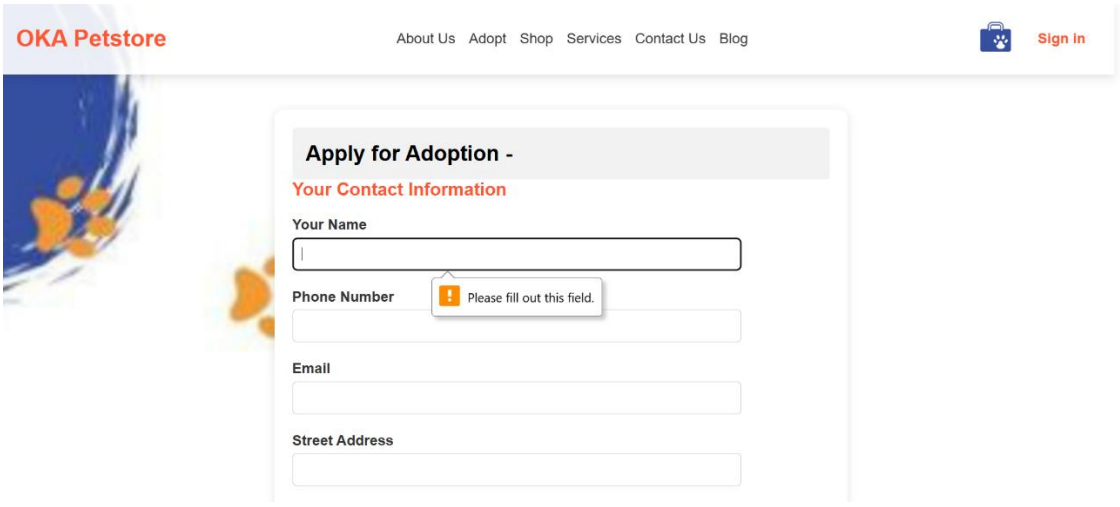


Figure 84 When a user tries to submit an adoption application and one of the fields is empty

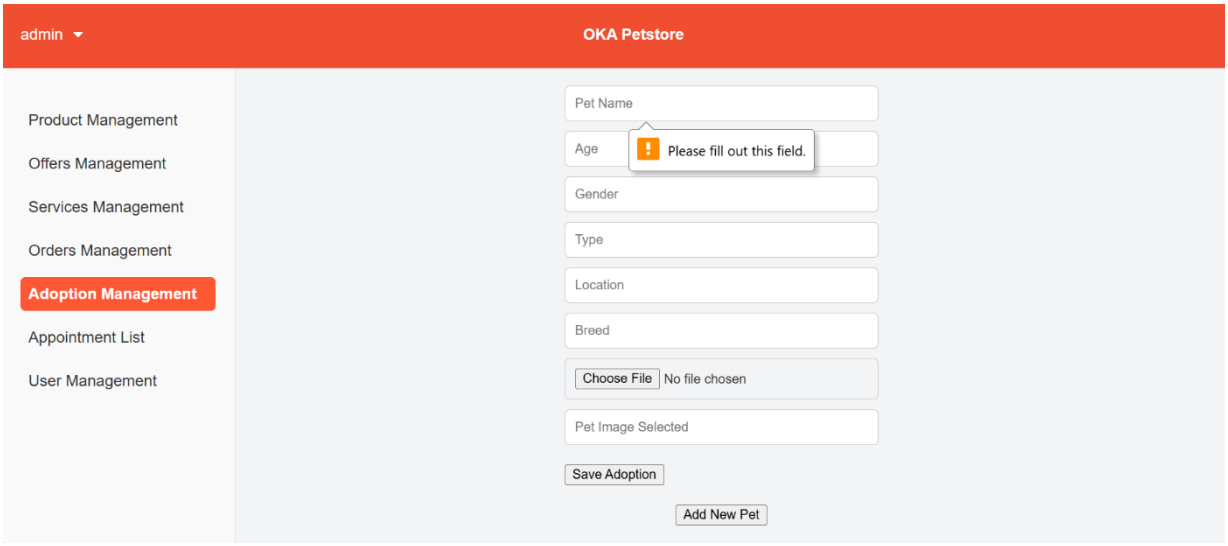


Figure 85 When a manager tries to add a pet for adoption and the fields are empty

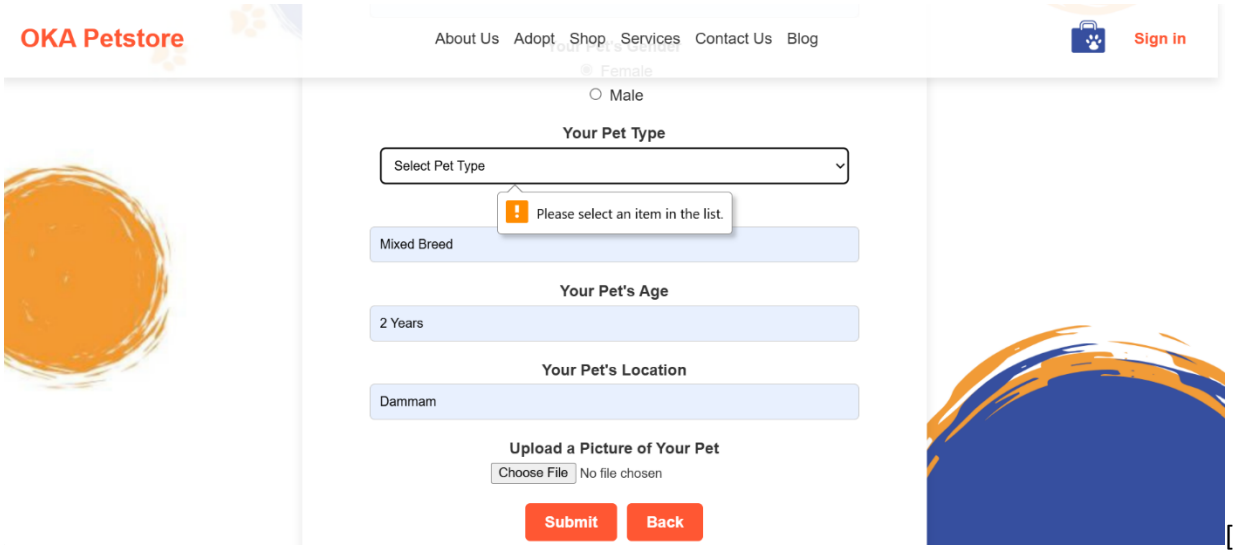


Figure 86 When a user tries to submit in find new home page without selecting all the fields

1.8 privecy and terms

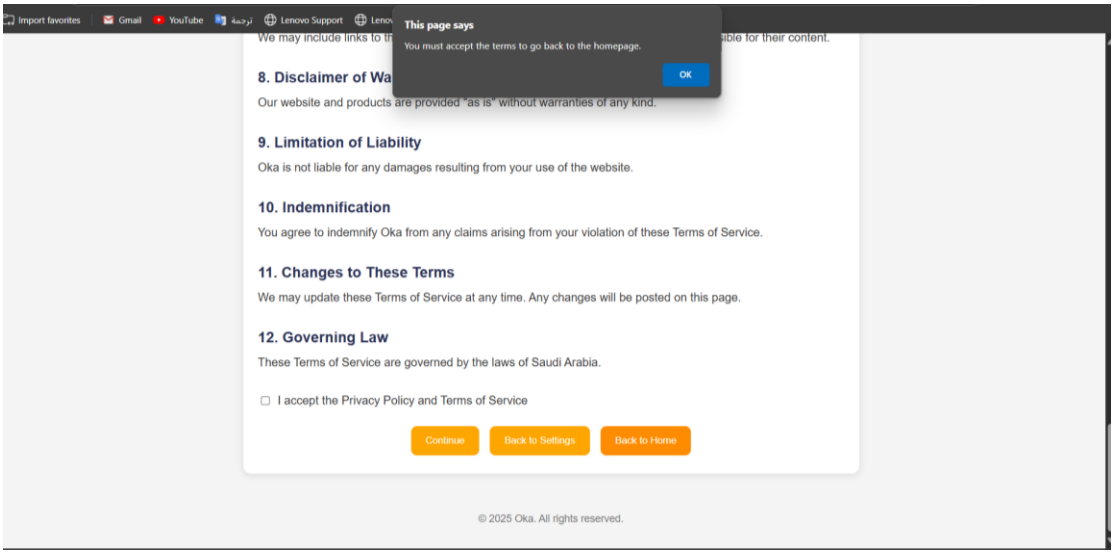


Figure 87 When a user tries to opt out the privacy and terms and continue

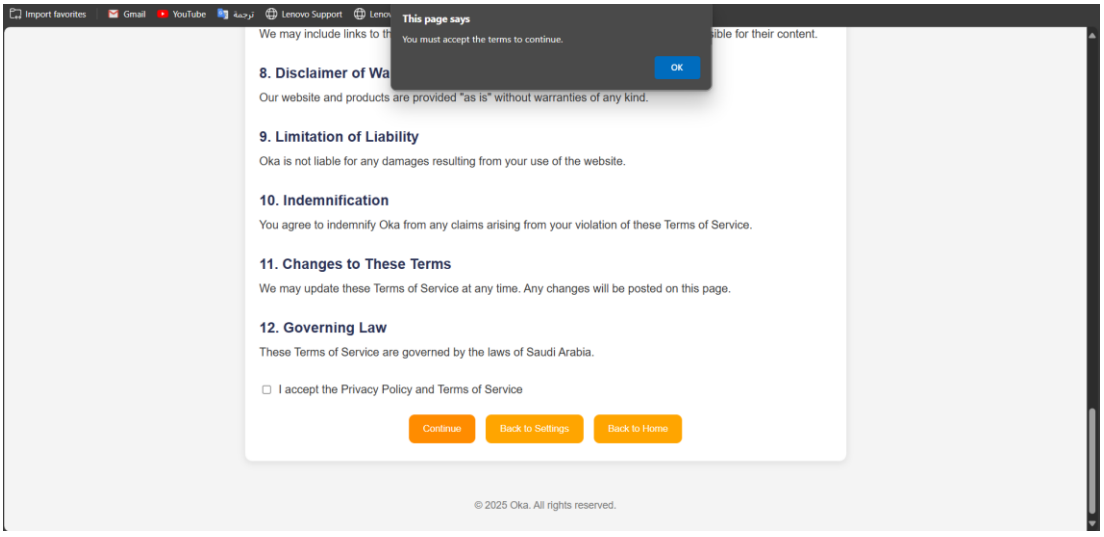


Figure 88 When a user tries to opt out the privacy and terms and back homepage